MyHealth Reference Manual v1.0.0

README
MyHealth 🔥 — Your Personal Fitness OS
🔭 Observability & analytics (all free tier, all opt-in)
🆕 Phase 1 – 6 — Guest Mode + Android + Wear OS + new domains
🧭 Apple-side architecture
📁 Project Structure
✨ MVP Features
🚀 Run It
🔌 API Reference
🌐 Data Sources (free, no API key)
🔒 Privacy posture (on-device AI)
📦 Bundle IDs (preserved)
🚧 Roadmap (post-MyHealth-v1)
CHANGELOG
Changelog
[Unreleased] - v1.4.0
Commit history (since repo creation)
DESIGN
MyHealth — Design & Architecture
1. Vision
2. Platform Map
3. Five-Layer Domain Architecture
4. Feature Catalogue
5. Data Flow
6. Tech Stack Summary
7. Repository Map
8. Privacy posture
MYHEALTH-FEATURES
MYHEALTH-FEATURES.md — v1.0 feature inventory
Cross-platform matrix
Onboarding & accounts
Health Tracking — Vitals (HealthKit / Health Connect)
Health Tracking — Sleep, recovery, mood
Health Tracking — Biological-age engine
Workouts — Training
Workouts — Media + condition-aware filtering
Diary, nutrition & activities
Medicine reminders
Health articles
Apple Watch surfaces

Wear OS surfaces
Cross-platform sync & portability
Backend API (Express + SQLite)
Settings
Telemetry & observability (all free tier, all opt-in)
Privacy posture
Distribution
NOT in v1 (explicitly)
OBSERVABILITY
MyHealth — Observability & Analytics Stack
1. The chosen stack
2. Where MyHealth currently lives on the spectrum
3. What's wired in this repo
4. Privacy contract — same on every platform
5. Event taxonomy (kept in sync across platforms)
6. Mixpanel / Amplitude alternative
7. Optional: full Datadog/New Relic/Grafana setup
8. Setup checklist
9. Honest limitations of this stack
PRIVACY
MyHealth — Privacy Policy
TL;DR
What MyHealth stores
What MyHealth sends off-device (only if you opt in)
What MyHealth never sends, ever
Third-party SDKs
Marketing site
Children
Your rights
Changes to this policy
Contact
PRODUCTION
MyHealth — Production Readiness & Store Deployment
1. Are you ready to ship today? — gap audit
2. Free deploy path — who pays for what
3. The wired-up CI/CD path (what you already have)
4. One-time setup checklist
5. Three-week pre-launch polish checklist
6. Things you do NOT need to worry about
7. The realistic answer to "is it production-ready?"
QUICKSTART
MyHealth — 5-Minute Quick-Start
🌐 Marketing site — 30 seconds
🔌 Backend (Express + SQLite) — 3 minutes

📱 Expo (cross-platform RN) — 5 minutes
🍎 iOS app — 5 minutes (macOS only)
⌚ watchOS app — 5 minutes (macOS only)
🤖 Android phone app — 5 minutes
⌚ Wear OS app — 5 minutes
🧪 Run every test on your machine — 1 line
🚀 Cut a release — 2 lines
🆘 Help

RELEASE-VIDEO-SCRIPT

Release Video Script — FitFusion

Hero — top of marketing site | 4

Features overview | 5

Highlight reel | 5

What's next | 5

Call to action — bottom of page | 4

RELEASING

MyHealth — Releasing & Distribution

1. TL;DR — Cut a release
2. Workflow files
3. Direct Google Play Store upload — yes, fully automated
4. Apple App Store — partial automation
5. Wear OS distribution
6. Versioning
7. Distribution checklist
8. First-time secrets setup
9. Local pre-release dry run

SENTRY

MyHealth — Sentry Crash Reporting

1. Free tier reality check
2. Privacy posture (what gets sent and what doesn't)
3. Wiring per platform
4. Setup steps (one-time per project)
5. User-visible toggle
6. Testing the wiring
7. What's NOT done

SIGNING

Signing — FitFusion

Android

iOS

Desktop

watchOS

STORE-PACKAGING

MyHealth — Store Packaging Reality Check

TL;DR

1. Apple Watch — must be embedded in the iPhone `.ipa`
2. Wear OS — two valid distribution shapes (we use shape A)
3. Updated `release.yml` matrix — what we *should* aim for
4. Action items before App Store submission
5. Action items before Play Store submission
6. Honest summary
Desktop binaries (Electron)
TESTING
MyHealth — Sanity Testing Guide
0. Prerequisites
1. Backend (any OS)
2. Marketing site (any OS, no install)
3. Expo cross-platform app (mobile or web)
4. iOS native app — simulator (Mac required)
5. watchOS app — simulator (Mac required)
6. Real-device-only features
7. Automated tests
8. Known caveats — read before reporting bugs
9. Issue template
10. Guest Mode (no-account / local-only)
11. Android phone app
12. Wear OS app
13. Automated CI
14. Local one-shot test runner
15. Local simulator / emulator launchers
16. Release dry-run
17. Direct Google Play upload
18. CI test strategy for FitFusionCore (audit summary)
19. Known CI caveats
WORKOUT-MEDIA
MyHealth — Workout Media + Condition-Aware Suggestions
1. Where the medical-decision line is drawn
2. Health conditions the user can declare (opt-in)
3. Workout filtering rules
4. Workout media (GIFs / videos)
5. Diet suggestions
6. iOS implementation
7. Android implementation
8. Doctor-disclaimer policy
9. What this is NOT

AMERICAN GROUP LLC

MyHealth

Reference Manual

Version 1.0.0

Generated 2026-05-08

README

README.md

MyHealth 🔥 — Your Personal Fitness OS

CI passing | Backend passing | iOS (simulator) passing | Android (instrumented) failing
Expo (mobile) passing | Marketing site (GitHub Pages) passing | codecov unknown

📖 **Docs**: QUICKSTART.md (5-min setup per platform) · DESIGN.md (architecture · features · data flow) · TESTING.md (sanity-test checklist · CI · simulator/emulator) · RELEASING.md (releases · GitHub binaries · Play Store · App Store) · PRODUCTION.md (production readiness · store deployment · gaps) · STORE-PACKAGING.md (watch-app bundling reality check) · SENTRY.md (crash reporting · privacy · setup) · OBSERVABILITY.md (full stack: Sentry + PostHog + Grafana free tier) · WORKOUT-MEDIA.md (workout GIFs · condition-aware filtering · diet suggestions · doctor disclaimer)

🔭 Observability & analytics (all free tier, all opt-in)

Concern	Tool	Free tier	Where it's wired
Crashes + APM + logs	**Sentry**	5K errors/mo	iOS · watchOS · Android · Wear · Expo · backend
Product analytics + feature flags + replays	**PostHog**	1M events/mo (OSS, EU region)	iOS · Android · Expo · backend
Server metrics (optional)	Grafana Cloud Free	10K series, 50 GB logs	docs only
Uptime monitoring (optional)	UptimeRobot	50 monitors	docs only

Privacy contract: every SDK is **off by default**. Users opt in via Settings → Privacy. Every wrapper strips `event.user` and never sends health data, meal contents, medicine names, photos, or screen recordings. Self-hostable if you outgrow the free tier. Mixpanel / Amplitude can be swapped in 1-line per platform.

Full setup + alternatives matrix: `OBSERVABILITY.md`.

> *Not just another workout app — a unified iPhone + Apple Watch + Android + Wear OS operating system for your body. Powered by on-device AI and end-to-end native.*

MyHealth combines **training, cardio, nutrition, sleep, mindfulness, on-device AI coaching, real-time workout mirroring, medicine reminders, biological-age estimation, and social challenges** into one cohesive experience built around a 5-layer architecture.

Now runs on 5 platforms — all with optional no-account "Guest Mode":

Platform	Tech	Status
📱 **iOS** (iPhone)	Swift + SwiftUI + HealthKit + CoreData/CloudKit	Full feature set
⌚ **watchOS** (Apple Watch)	Swift + SwiftUI + HKWorkoutSession + WCSession	9 vertical tabs incl. Anatomy
🤖 **Android** (phone)	Kotlin 2.0 + Compose + Room + Health Connect + ML Kit + Hilt	Phase 4 — full bottom-nav port
⌚ **Wear OS** (Android Watch)	Kotlin + Wear Compose + Health Services + Tile + Complication	Phase 5 — 9 vertical pages incl. Anatomy
🌐 **Web / Expo** (Android + iOS + Web)	React Native (Expo)	Login + Guest button

Plus the public-facing **marketing website** (`index.html` + `styles.css` + `script.js`) at the root.

🆕 Phase 1 – 6 — Guest Mode + Android + Wear OS + new domains

This iteration added:

- **Guest Mode globally** (iOS + watchOS share `AuthStore.continueAsGuest()` ; Expo Login has a "Continue as Guest" button; Android `SettingsRepository.isGuest = true` by default). Any user can launch the app and start using it **without an email**. Existing JWT login path is preserved.
- **Onboarding flow** — 4-page Welcome → Profile setup → Goal → Done; gated by `UserDefaults.didOnboard` on iOS and `DataStore` on Android.
- **Medicine reminders** with `UNUserNotificationCenter` (iOS) and `AlarmManager` + WorkManager (Android). Take / Snooze 10 min actions, persisted dose log, 14-day adherence streak, archive.
- **Food diary** (iOS + Android Room) — daily macro rings + 14-day history + custom-meal builder.
- **Activities** (non-workout movement: walking, gardening, cleaning) with full add/edit/delete.
- **Health articles** — bundled offline-readable `HealthArticleSeed` + live MyHealthfinder topics + OpenFDA drug-info lookup.
- **Cross-platform JSON schema** at `shared/schemas/myhealth.schema.json` — both Apple Core Data and Android Room entities map 1:1; export/import via `PortabilityService` so users can move profiles between iOS and Android.
- **Android phone app** under `android/` — native Kotlin + Compose + Material 3 dynamic theming + 5 localizations.
- **Wear OS app** under `android/wear/` — vertically-paged tabs, Tile + Complication for readiness, Health Services for live HR.
- **`/api/medicine/lookup`** — public no-auth route for OpenFDA drug labels.

🧭 Apple-side architecture

```
┌──────────────────────┐  ⇄  ┌──────────────────────┐  ⇄  ┌──────────────────────┐
│ 📱 iOS app (ios/)    │     │ ⌚ watchOS app       │     │ ☁️  CloudKit         │
│ SwiftUI · 5+1 tabs   │     │ (watch/) SwiftUI     │     │ iCloud.com.fitfusion │
│ Home · Train · Diary │     │ Live workout · Run   │     │ Plans · Meals · Mood │
│ Sleep · More         │     │ Anatomy · Crown log  │     │                      │
└──────────────────────┘     └──────────────────────┘     └──────────────────────┘
          │                            │
          │   shared/FitFusionCore     │
          │   (Models · APIClient ·    │
          │    AuthStore (+ Guest) ·   │
          │    CloudStore (15 entities))
          ▼                            ▼
   ┌──────────────────────────────────────────────┐
   │ 🩺 HealthKit  +  🏋️ WorkoutKit (Apple SDKs)  │
   │ HRV · sleep stages · workouts · macros · etc.│
   └──────────────────────────────────────────────┘

   ┌──────────────────────────────────────────────┐
   │ 🌐 Express + SQLite backend (server/)        │
   │ Auth · Profile · Metrics · Insights ·        │
   │ Nutrition · Live public health data          │
   └──────────────────────────────────────────────┘
                      ▲
                      │ REST (cross-platform)
   ┌──────────────────────────────────────────────┐
   │ 📱 Expo React Native app (mobile/)           │
   │ Android / iOS / Web fallback                 │
   └──────────────────────────────────────────────┘
```

📁 Project Structure

```
FitFusion/
├── index.html                    ← Marketing website (rebranded to FitFusion)
├── styles.css
├── script.js
├── server/                       ← Node.js + Express backend
│   ├── server.js, db.js
│   ├── middleware/auth.js
│   ├── routes/{auth,profile,health,nutrition,insights}.js
│   └── package.json
├── shared/
│   └── FitFusionCore/            ← Shared Swift Package (iOS + watchOS)
│       ├── Package.swift
│       └── Sources/FitFusionCore/
│           ├── Models.swift
│           ├── APIClient.swift
│           ├── AuthStore.swift
│           ├── CloudStore.swift
│           ├── WatchConnectivity/Bridge.swift
│           └── FitFusionModel.xcdatamodeld
├── ios/                          ← Native SwiftUI iOS app (NEW)
│   ├── project.yml               (XcodeGen)
│   └── FitFusion/
│       ├── FitFusionApp.swift, RootView.swift
│       ├── Info.plist, FitFusion.entitlements
│       ├── Views/{Auth,Home,Train,Run,Nutrition,Sleep}/
│       └── Services/{iOSHealthKitManager, ReadinessEngine,
                  RecoveryService, NutritionService,
                  WorkoutScheduler, AppIntents}.swift
├── watch/                        ← SwiftUI watchOS app
│   ├── project.yml
│   ├── HealthAppWatch/           (uses FitFusionCore)
│   └── FitFusionComplication/    (Readiness on watch face — NEW)
└── mobile/                       ← Expo React Native (cross-platform)
    └── ...
```

✨ MVP Features

1. Workout Library 🏋️

Browse strength · cardio · yoga · mobility · beginner / advanced — tap any workout and “Send to Watch” via WorkoutKit; it appears in the native Watch Workout app.

2. Run Tracker 🏃

Live pace + distance on the Watch (CMPedometer); route maps + pace/elevation charts on the iPhone (MapKit + Swift Charts).

3. Food & Calorie Logging 🥗

Barcode scanning (VisionKit `DataScannerViewController`) → Open Food Facts lookup → macros written to HealthKit as an `HKCorrelation` and synced via CloudKit + the Express backend.

4. Sleep & Recovery 💤

Last-night sleep stages chart, 7-day HRV trend, 0–100 recovery score with traffic-light zones, and a Wind Down sheet with a guided breath / mindful session.

5. Apple Watch Live Workout Controls ⌚

`HKWorkoutSession` + `HKLiveWorkoutBuilder` with always-on display, HR zones, calories, elapsed time. Run mode adds live pace.

6. HealthKit Sync 🩺

The iOS app's `iOSHealthKitManager` requests authorization for the full read set (steps, HR, HRV, RHR, sleep stages, dietary energy/protein/carbs/fat, water, mindful, workouts) and write set (water, body mass, dietary correlations, mindful sessions, workouts). The watchOS app's `HealthKitManager` mirrors this and writes back to HealthKit on every wrist log.

🚀 Run It

1. Backend

```
cd server
cp .env.example .env       # set JWT_SECRET to something random
npm install
npm run dev                 # → http://localhost:4000
```

Test:

```
curl http://localhost:4000/api/health-check
```

2. iOS app

```
brew install xcodegen
cd ios
xcodegen generate
open FitFusion.xcodeproj
```

Pick an iPhone simulator (iOS 17+) and hit ▶️.

3. watchOS app

```
cd watch
xcodegen generate
open HealthAppWatch.xcodeproj
```

Pick an Apple Watch simulator and hit ▶️. See `watch/README.md`.

4. Mobile (Expo, cross-platform)

```
cd mobile
npm install
npm start
```

🔌 API Reference

Method	Path	Auth	Purpose
GET	`/api/health-check`	–	Liveness check
POST	`/api/auth/register`	–	Create account
POST	`/api/auth/login`	–	Get JWT
GET	`/api/profile`	✅	User + profile + BMI
PUT	`/api/profile`	✅	Upsert profile fields
POST	`/api/profile/metrics`	✅	Log a metric
GET	`/api/profile/metrics?type=weight`	✅	List metrics
POST	`/api/nutrition/meal`	✅	Log a meal (kcal + macros)
GET	`/api/nutrition/today`	✅	Today's meals + totals
GET	`/api/insights/readiness`	✅	0–100 readiness score
GET	`/api/insights/weekly`	✅	7-day per-metric aggregates
GET	`/api/health/topics?keyword=sleep`	–	Live MyHealthfinder topics
GET	`/api/health/topic/:id`	–	Full topic content
GET	`/api/health/drug?name=ibuprofen`	–	Open FDA drug labels

🌐 Data Sources (free, no API key)

- **MyHealthfinder** — health.gov
- **Open FDA** — open.fda.gov
- **Open Food Facts** — world.openfoodfacts.org

🔒 Privacy posture (on-device AI)

All model inputs and outputs stay on the device. The Core ML `AdaptivePlanner` runs locally with `MLPredictionOptions(usesCPUOnly: false)`; nightly `MLUpdateTask` fine-tuning runs locally via

`BGTaskScheduler` with the user's recent (HRV, sleep, planned vs actual workout, perceived exertion) tuples. The Vision meal-photo classifier and nutrition-label OCR also run locally. Only the chosen `WorkoutTemplate` ID is ever sent to the backend (just as today).

📦 Bundle IDs (preserved)

Despite the rebrand, bundle IDs (`com.fitfusion.ios` , `com.fitfusion.watch` , `com.fitfusion.watch.complication`), the App Group (`group.com.fitfusion`), and the CloudKit container (`iCloud.com.fitfusion`) **remain unchanged** so already-synced CloudKit data survives. Only display names, marketing copy, and Siri phrase wording change.

🚧 Roadmap (post-MyHealth-v1)

- SensorKit deep-context (requires separate Apple entitlement approval)
- Apple Game Center leaderboards (`LeaderboardClient` flag, off by default)
- Apple Vision Pro spatial coach view
- Offline workout playback (downloaded media)

CHANGELOG

CHANGELOG.md

Changelog

All notable changes to this project will be documented in this file.

This project adheres to Semantic Versioning.

[Unreleased] - v1.4.0

Initial public release. See distribution/whatsnew/v1.4.0/en-US.txt for the user-facing summary.

Commit history (since repo creation)

- 20650c0 Initial commit (Srikanth Patchava, 2026-05-01)
- 6468f05 feat: full MyHealth platform ΓÇö iOS + watchOS + Android + Wear OS + Expo + backend (Srikanth Patchava, 2026-05-01)
- 7061743 fix(ci): trigger workflows on master, fix release.yml secret-gating, drop missing npm cache (Srikanth Patchava, 2026-05-01)
- e154d1a fix(ci): macOS in Package.swift, Jest setupFiles key, Pages enablement, Android adaptive icon (Srikanth Patchava, 2026-05-01)
- fbd9d5e fix(ci): repair 47 Swift escapes, mobile/package.json JSON, marketing Pages tolerance (Srikanth Patchava, 2026-05-01)
- 3e750d6 fix(swift): combine surrogate pair into single scalar in BiologicalAgeEngine (Srikanth Patchava, 2026-05-01)
- 04e7888 fix(swift): gate BackgroundTasks usage with os(iOS) instead of canImport (BGTaskScheduler is iOS-only) (Srikanth Patchava, 2026-05-01)
- b69b28f ci: test FitFusionCore on iOS Simulator (UIKit + WatchConnectivity preclude macOS host) (Srikanth Patchava, 2026-05-01)
- 0558200 Enhance with vibrant colors, animated gradients, full SEO + CI/CD deploy (Srikanth Patchava, 2026-05-03)
- 1aefe2c Add Google Analytics GA4 tracking (G-Y0CDD6QJ9J) (Srikanth Patchava, 2026-05-03)
- 4085759 Add sitemap.xml and robots.txt for SEO crawling (Srikanth Patchava, 2026-05-03)
- 9db9f20 fix(android): drop self-referential Modifier.clickable / Modifier.clip extensions (Srikanth Patchava, 2026-05-04)
- 1346552 fix(wear): add Guava + use DimensionBuilders.expand() in ReadinessTileService (Srikanth Patchava, 2026-05-04)
- 008849c feat(release): TestFlight auto-upload + PRODUCTION.md gap audit (Srikanth Patchava, 2026-05-04)
- 7706fce fix(release): publish-github-release tolerates partial build failures + always includes source archives (Srikanth Patchava, 2026-05-04)
- e12b36b fix(release): produce real binaries for every platform; drop source-archive fallback (Srikanth Patchava, 2026-05-04)

- 0a16c30 feat(ci): combined Pre-Release Tests gate + platform-clear binary names (Srikanth Patchava, 2026-05-04)
- e9c31fa docs(stores): STORE-PACKAGING.md ΓÇö watch-app bundling reality + remaining gaps (Srikanth Patchava, 2026-05-04)
- 613d1e0 feat: Sentry crash reporting wired across iOS+watchOS, Android phone+wear, Expo, backend (Srikanth Patchava, 2026-05-04)
- 4c29287 feat(observability): PostHog product analytics + OBSERVABILITY.md (Sentry + PostHog + Grafana matrix) (Srikanth Patchava, 2026-05-04)
- 5155221 ci(release): pass Sentry + PostHog secrets through to build steps (Srikanth Patchava, 2026-05-04)
- 4fb5c8b fix(ci): regenerate server lockfile with @sentry/node + posthog-node; mark wear-ui-tests informational (Srikanth Patchava, 2026-05-04)
- 723c4f4 fix(ci): emulator-based UI tests informational; release flatten only ships binaries (Srikanth Patchava, 2026-05-04)
- 0c883c7 feat(health): condition-aware workout filtering + GIF demos + diet suggestions (Srikanth Patchava, 2026-05-04)
- 88c81af docs(stores): correct xcodegen syntax for Apple Watch embed migration (Srikanth Patchava, 2026-05-04)
- aa19f23 fix(release): unmask iOS+Android phone build errors; drop xcodegen capabilities/INFOPLIST_KEY blocks (Srikanth Patchava, 2026-05-04)
- 6f8ce0d fix(release): drop GCC_PREPROCESSOR_DEFINITIONS injection that broke xcodebuild arg parsing (Srikanth Patchava, 2026-05-04)
- e2c6423 fix(ci): xcode-select to latest Xcode in every macOS job (objectVersion 77 fix) (Srikanth Patchava, 2026-05-04)
- 9cc5d88 fix(release): repair iOS/watchOS build failures (Srikanth Patchava, 2026-05-06)
- 2d2da82 chore(release): bump to 1.3.3 (Srikanth Patchava, 2026-05-06)
- 5652e6f fix(release): build iOS/watchOS sim binaries in Debug to match pre-release-tests (Srikanth Patchava, 2026-05-06)
- 2fd2195 docs: add MIT LICENSE and PRIVACY.md (Phase 2 production readiness) (Srikanth Patchava, 2026-05-06)
- 08a3d0d docs(distribution): add SUBMISSION-CHECKLIST.md (Phase 3 production readiness) (Srikanth Patchava, 2026-05-06)

DESIGN

DESIGN.md

MyHealth — Design & Architecture

> Complete architectural reference for MyHealth across **iOS · watchOS · Android phone · Wear OS · Web (Expo) · Express backend**.

This document is for engineers and designers joining the project. For end-user docs see `README.md`. For sanity-test walkthroughs see `TESTING.md`. For release / deploy procedure see `RELEASING.md`.

---

1. Vision

> **Your personal fitness OS, on every device, with no email required.**

MyHealth is a privacy-first, local-first fitness + health platform. Anyone can launch the app and start using it in 60 seconds — no account, no email, no "create your profile in the cloud" wall. Cloud sync is **opt-in** and only turns on if the user explicitly signs in.

Core promise: every personal data point (meals, workouts, mood, vitals, medicines, biological-age inputs) lives **on the user's device** by default. Nothing leaves unless they opt in. AI inference (workout planner, meal-photo classifier, label OCR, biological-age engine) runs on-device only.

---

2. Platform Map

```
                                  ┌──────────────────────────────────────┐
                                  │ Public docs + reference content      │
                                  │ health.gov MyHealthfinder · OpenFDA  │
                                  │ Open Food Facts · USDA FDC           │
                                  └──────────────────────────────────────┘
                                                │  (no auth required)
                                                ▼
┌──────────────────────┐    ┌──────────────────────────────────┐
┌──────────────────┐
│  iOS app (Swift)     │    │  Android app (Kotlin + Compose)  │    │  Expo (RN)
│                  │
│  + watchOS companion │◄──►│   + Wear OS companion            │◄──►│  Web ·
Android · iOS│
│  CoreData + CloudKit │    │  Room + DataStore + Health Connect│    │
AsyncStorage      │
│  HealthKit + WorkoutKit│  │  CameraX + ML Kit + WorkManager  │    │
│                  │
└──────────────────┘    └──────────────────────────────────┘
└────────┬─────────┘
         │                                  │                                  │
         │            (optional sync — only when user signs in)                │
         └──────────────────────────────────┴──────────────────────────────────┘
                                  ▼
                        ┌──────────────────────┐
                        │ Express + SQLite     │
                        │ (port 4000)          │
                        │ Auth · Metrics ·     │
                        │ Meals · Insights ·   │
                        │ Social · Medicine    │
                        └──────────────────────┘
```

Surface	Tech	Local store	Online dependencies (optional)
iOS app	Swift + SwiftUI 5	Core Data + iCloud	Express backend (sign-in only)
watchOS app	Swift + SwiftUI	Shared CoreData + WCSession	iOS app (paired)
Android app	Kotlin 2.0 + Compose + Material 3	Room + DataStore	Express backend (sign-in only)
Wear OS app	Kotlin + Wear Compose	SharedPreferences	Phone app (paired)
Web/Expo	React Native (Expo)	AsyncStorage	Express backend
Backend	Node.js + Express + better-sqlite3 + JWT	SQLite file	OpenFDA · MyHealthfinder · Open Food Facts

---

3. Five-Layer Domain Architecture

```
LAYER 5 — SOCIAL & GAMIFICATION
Friends · Challenges · Leaderboards · Badges · Streaks
SharePlay shared workouts · MSMessage activity cards

                          ▲

LAYER 4 — CLOUD + SURFACES
CloudKit (Apple) · Live Activities · Widgets · Mirroring
Compose Tile + Complication (Wear) · WorkManager

                          ▲

LAYER 3 — ON-DEVICE AI
AdaptivePlanner · PersonalFineTuner · BiologicalAgeEngine
MealPhotoRecognizer (Vision · ML Kit) · NutritionLabelOCR

                          ▲

LAYER 2 — PLATFORM SDK CORE
HealthKit + WorkoutKit (Apple) · Health Connect (Android)
CoreData · Room · DataStore · UserNotifications · AlarmManager

                          ▲

LAYER 1 — RAW SENSING
Watch live HR/HRV · GPS · Workout sessions · Steps · SpO₂
Camera (barcode + meal photo + nutrition label OCR)
```

---

4. Feature Catalogue

4.1 Onboarding & profile (every platform)

- 4-page first-launch flow: Welcome → Profile setup (name, DOB, sex, height, weight, units) → Goal (lose/maintain/build/endurance/general) → Done.
- Profile persisted to a single `ProfileEntity` row (Core Data on iOS, Room on Android).
- Editable any time via Settings → Profile.

4.2 Guest Mode (everywhere)

- iOS / watchOS / Web: `AuthStore.continueAsGuest()` flips `isAuthenticated = true` and `isGuest = true` without contacting the backend.
- Android: `SettingsRepository.isGuest = true` by default — no login screen at all.

- `APIClient.send(...)` short-circuits authenticated routes when `isGuest`, returning local stub responses so the existing UI keeps working.
- Public-API routes (MyHealthfinder, OpenFDA, Open Food Facts) still go to the network so guests still benefit.
- "Sign in for cloud sync" remains a one-tap upgrade in Settings.

4.3 Training (Apple-side rich, Android-side ported)

- **Anatomy picker** — front/back body silhouette (iOS) or region tabs (Watch / Android), drilling into muscle → exercise list → exercise detail.
- **Exercise library** — 30+ strength lifts, 3 cardio modalities, 10 stretches with steps + form tips. Bundled in Swift `ExerciseLibrary.swift` and Kotlin `Exercises.kt` (1:1 data parity).
- **Pre-built programs** — Push/Pull/Legs, Upper/Lower, Full Body 3×, Beginner Strength.
- **Workout logger** — sets / reps / weight, persisted to CloudKit-synced `ExerciseLogEntity`. Surfaces personal records.
- **Rest timer** with circular countdown (iOS).
- **Custom workout builder** — drag-reorderable exercise list, saved to CloudKit.

4.4 Diary, nutrition & activities

- **Food diary** — daily macro rings + 14-day history + custom-meal builder.
- **Snap Meal (iOS)** — Vision / ML Kit on-device food classification → Open Food Facts macro lookup.
- **Read Label (iOS / Android)** — Vision / ML Kit text recognition → kcal/protein/carbs/fat regex extraction.
- **Barcode scanner** — VisionKit / ML Kit barcode → Open Food Facts.
- **Activities** — non-workout movement: walking, gardening, cycling around town. Add / Edit / Delete.

4.5 Sleep, recovery, mood

- HealthKit / Health Connect sleep stages (REM/Deep/Core/Awake).
- Recovery score 0–100 from sleep + HRV + RHR.
- 5-emoji mood logger.
- 2-axis HKStateOfMind logger (iOS 17+).
- Wind Down sheet with breath/mindful session.

4.6 Vitals (full HealthKit / Health Connect coverage)

- **Cardiovascular** — HR, RHR, HRV, SpO_2, VO_2 Max, ECG count, irregular rhythm, respiratory rate.
- **Body composition** — weight, BMI, body fat %, lean mass.
- **Sensor / manual** — blood pressure, blood glucose, body temp.
- **Activity** — steps, distance, active + basal calories, exercise minutes, floors.
- **Sleep** — total + REM/Deep, wrist temp delta.
- **Environmental** — noise (dB), UV, handwashing, walking steadiness.
- Honest disclaimers in UI for non-sensorable items (body water %, hydration sensor, snoring, continuous BP).

4.7 Biological Age engine

- On-device, no model file, pure Swift / pure Kotlin port.
- Inputs: chronological age, sex, RHR, HRV, VO_2Max, sleep, BMI, body fat %, BP, weekly exercise min, daily steps, smoker / heavy-alcohol flags.
- Output: chronological vs biological age + per-factor ± year breakdown + confidence (scales with how many signals are present).
- Verdict line ("Younger than your age ✨" / "Right on track" / etc.).

4.8 Medicine reminders

- iOS: `UNUserNotificationCenter` with notification category `MEDICINE_REMINDER` and two actions (Take / Snooze 10 min).
- Android: `AlarmManager` (`setRepeating`) + `BroadcastReceiver` with the same Take / Snooze actions; `BootReceiver` re-arms after reboot.
- Persisted dose log + 14-day adherence streak + archive.
- Cron-like schedule (times of day × weekdays) stored as JSON inside the entity for easy cross-platform export.
- Optional OpenFDA drug-info lookup when adding a new medicine.

4.9 Health articles

- Bundled `HealthArticleSeed` (works offline / guest mode) — 6 starter articles.
- Live MyHealthfinder topics from `/api/health/topics` (no auth required).
- OpenFDA drug-label search via `/api/medicine/lookup` (no auth required).

4.10 Watch / Wear OS

- Vertically-paged tabs: Quick Log → Live Workout → Run → **Anatomy** → Water → Weight → Mood → History → Settings.
- Apple Watch: HKWorkoutSession, GPS routes, Workout Mirroring sender, Live Activity, complications.
- Wear OS: Health Services for live HR + GPS, Tile + Complication for Readiness.

4.11 Cross-platform portability

- Single canonical schema at `shared/schemas/myhealth.schema.json` .
- iOS `PortabilityService.exportEverything()` writes a date-stamped JSON file to share via AirDrop / Files.
- Android side will mirror this via `data/portability/PortabilityService.kt` (export TODO).
- Erase-all-on-device action wipes every entity and resets onboarding.

---

5. Data Flow

5.1 Adding a meal (iOS, guest mode)

```
User taps "+" in Diary
        │
        ▼
NutritionView → FoodSearchView (Open Food Facts query, no auth)
        │
        ▼
SelectFood → CloudStore.addMeal( … )
        │
        ▼
NSManagedObjectContext.save() → CoreData → (iCloud sync if account)
        │
        ▼
HomeDashboardView observes today's meals → ring updates
```

5.2 Medicine reminder firing (iOS)

```
Cold start → MedicineReminderService.bootstrap() registers category
            │
            └─ FitFusionApp .task → resyncAll() reschedules every active medicine
                                                                  │
                                                                  ▼
9:00 AM today                                                     │
            │                                                     │
            ▼                                                     │
UNCalendarNotificationTrigger fires ◀──────────────────────────────┘
            │
            ▼
Banner shows with Take / Snooze 10 min actions
            │
            ▼
User taps Take → MedicineReminderService.handleAction("MED_TAKE", info)
            │
            ▼
CloudStore.logDose(medicineId, takenAt: now)
            │
            ▼
MedicineDetailView observes → adherence streak ticks up
```

5.3 Biological-age estimation (any platform)

```
VitalsView surfaces the snapshot ← VitalsService.refresh()
                                          │
                                          ▼
                            Read every HealthKit type into
                            a single VitalsSnapshot struct
                                          │
                                          ▼
User taps "Biological Age" → BiologicalAgeView
                                          │
                                          ▼
BiologicalAgeEngine.estimate(inputs)
                                          │
                                          ▼
Per-factor delta-years list + total bio age
                                          │
                                          ▼
Side-by-side gauge + sortable factor breakdown rendered on-device
```

---

6. Tech Stack Summary

Concern	iOS / watchOS	Android / Wear	Web (Expo)	Backend
UI	SwiftUI 5	Jetpack Compose + Material 3	React Native	–
Local store	Core Data + CloudKit	Room	AsyncStorage	SQLite (better-sqlite3)
Settings	UserDefaults	DataStore	AsyncStorage	env vars
Health platform	HealthKit + WorkoutKit	Health Connect + Health Services	–	–
Live HR / workouts	HKWorkoutSession + WCSession	Health Services + ExerciseClient	–	–
Notifications	UNUserNotificationCenter	NotificationManager + AlarmManager	–	–
Background work	BGTaskScheduler	WorkManager	–	–
Vision / ML	Vision + Core ML	ML Kit + TensorFlow Lite	–	–
Camera	VisionKit	CameraX	–	–
HTTP	URLSession	Ktor	fetch	–
Auth	JWT (optional)	JWT (optional)	JWT (optional)	bcryptjs + jsonwebtoken
Tests	XCTest	JUnit + Truth + Robolectric + Compose UI	Jest	Jest + supertest
Build	XcodeGen	Gradle 8 + Kotlin 2.0	Expo CLI	npm

---

7. Repository Map

```
HealthApp/
├── README.md                       ← User-facing overview
├── DESIGN.md                       ← This document
├── TESTING.md                      ← Sanity-test checklist
├── RELEASING.md                    ← Release procedure + Play Store
├── index.html, styles.css, script.js   ← Marketing site
├── shared/
│   ├── FitFusionCore/              ← Swift Package, iOS + watchOS
│   └── schemas/
│       └── myhealth.schema.json    ← Canonical cross-platform JSON
├── ios/
│   ├── FitFusion/                  ← Main iOS app
│   ├── MyHealthLiveActivity/       ← Lock-screen Live Activity ext.
│   ├── MyHealthWidget/             ← Home/Lock widget ext.
│   ├── MyHealthMessages/           ← iMessage app ext.
│   └── project.yml                 ← XcodeGen
├── watch/
│   ├── HealthAppWatch/             ← watchOS app
│   └── FitFusionComplication/
├── android/
│   ├── settings.gradle.kts         ← Multi-module Gradle root
│   ├── core/                       ← Shared Kotlin: Models, Exercises, BioAge
│   ├── app/                        ← Phone app (Compose + Hilt + Room)
│   └── wear/                       ← Wear OS app (Wear Compose)
├── mobile/                         ← Expo (Android + iOS + Web)
├── server/                         ← Express + SQLite + JWT
└── .github/workflows/              ← CI / Release pipelines
```

---

8. Privacy posture

- All AI inference is on-device (Core ML, Vision, ML Kit, BiologicalAgeEngine).
- Guest Mode never sends personal data to any server. Public APIs (MyHealthfinder, OpenFDA, Open Food Facts) accept anonymous queries with no PII.
- Bundle ID `com.fitfusion.*` and CloudKit container `iCloud.com.fitfusion` are intentionally retained so existing CloudKit data survives the rebrand.
- HealthKit / Health Connect respect platform-level user permission gates.
- Backend uses bcrypt-hashed passwords + JWT; tokens never leave AsyncStorage / UserDefaults / DataStore on the client.
- Erase-all-data action wipes every entity from the local DB and resets onboarding.

MYHEALTH-FEATURES

MYHEALTH-FEATURES.md

MYHEALTH-FEATURES.md — v1.0 feature inventory

> Synthesized from `README.md`, `DESIGN.md`, and `WORKOUT-MEDIA.md`. The customer-facing promise is "your personal fitness OS, on every device, with no email required."

Cross-platform matrix

Surface	Tech	Status
📱 iOS (iPhone)	Swift + SwiftUI + HealthKit + CoreData/CloudKit	Full feature set
⌚ watchOS (Apple Watch)	Swift + SwiftUI + HKWorkoutSession + WCSession	9 vertical tabs incl. Anatomy
📟 Android (phone)	Kotlin 2.0 + Compose + Room + Health Connect + ML Kit + Hilt	Full bottom-nav port
⌚ Wear OS	Kotlin + Wear Compose + Health Services + Tile + Complication	9 vertical pages incl. Anatomy
🌐 Expo (Android + iOS + Web)	React Native	Login + Guest button
🌐 Marketing site	Static HTML/CSS/JS	GitHub Pages

Onboarding & accounts

- ☑ 4-page first-launch flow: Welcome → Profile (name, DOB, sex, height, weight, units) → Goal → Done.
- ☑ Goal picker: lose / maintain / build / endurance / general.
- ☑ Onboarding gated by `UserDefaults.didOnboard` (iOS) / `DataStore` (Android).
- ☑ **Guest Mode globally** — iOS / watchOS / Web `AuthStore.continueAsGuest()`; Android `SettingsRepository.isGuest = true` by default. Launch and use without an email.
- ☑ Existing JWT login path preserved as a one-tap upgrade ("Sign in for cloud sync").
- ☑ `APIClient.send(...)` short-circuits authenticated routes when guest, returning local stub responses.

Health Tracking — Vitals (HealthKit / Health Connect)

- ☑ **Cardiovascular** — HR, RHR, HRV, SpO_2, VO_2 Max, ECG count, irregular rhythm, respiratory rate.
- ☑ **Body composition** — weight, BMI, body fat %, lean mass.
- ☑ **Sensor / manual** — blood pressure, blood glucose, body temp.

- ☑ **Activity** — steps, distance, active + basal calories, exercise minutes, floors.
- ☑ **Sleep** — total + REM/Deep/Core/Awake stages, wrist temp delta.
- ☑ **Environmental** — noise (dB), UV, handwashing, walking steadiness.
- ☑ Honest disclaimers in UI for non-sensorable items (body water %, hydration sensor, snoring, continuous BP).

Health Tracking — Sleep, recovery, mood

- ☑ HealthKit / Health Connect sleep stages chart.
- ☑ 0–100 recovery score from sleep + HRV + RHR with traffic-light zones.
- ☑ 7-day HRV trend.
- ☑ 5-emoji mood logger.
- ☑ 2-axis HKStateOfMind logger (iOS 17+).
- ☑ Wind Down sheet with guided breath / mindful session.

Health Tracking — Biological-age engine

- ☑ On-device, no model file — pure Swift / pure Kotlin port.
- ☑ Inputs: chronological age, sex, RHR, HRV, VO_2Max, sleep, BMI, body fat %, BP, weekly exercise min, daily steps, smoker / heavy-alcohol flags.
- ☑ Output: chronological vs biological age + per-factor ± year breakdown + confidence (scales with available signals).
- ☑ Verdict line ("Younger than your age ✨" / "Right on track" / etc.).

Workouts — Training

- ☑ **Anatomy picker** — front/back body silhouette (iOS) or region tabs (Watch / Android), drilling into muscle → exercise list → exercise detail.
- ☑ **Exercise library** — 30+ strength lifts, 3 cardio modalities, 10 stretches with steps + form tips. 1:1 parity between Swift `ExerciseLibrary.swift` and Kotlin `Exercises.kt`.
- ☑ **Pre-built programs** — Push/Pull/Legs, Upper/Lower, Full Body 3×, Beginner Strength.
- ☑ **Workout logger** — sets / reps / weight, persisted to `ExerciseLogEntity`. Surfaces personal records.
- ☑ **Rest timer** with circular countdown (iOS).
- ☑ **Custom workout builder** — drag-reorderable exercise list, saved to CloudKit.
- ☑ **Send to Watch** via WorkoutKit — workout appears in the native Watch Workout app.
- ☑ **Run Tracker** — live pace + distance on Watch (CMPedometer); route maps + pace/elevation charts on iPhone (MapKit + Swift Charts).

Workouts — Media + condition-aware filtering

- ☑ Exercise GIFs / images served from GitHub Pages (`assets/exercises/<id>.gif`), lazy-loaded, graceful SF Symbol / Material icon fallback on 404.
- ☑ `EXERCISE_MEDIA_BASE_URL` build-time override for private CDN.
- ☑ **Health-conditions store** (opt-in, on-device only, never sent to server, never logged via analytics) — Cardiovascular, Metabolic, Respiratory, Musculoskeletal, Pregnancy / Kidney / Liver / Anemia.
- ☑ `ExerciseLibrary.recommended(for: userConditions)` filters out cautioned exercises and boosts beneficial ones (e.g. hides Back Squat / Deadlift for hypertension, promotes Cat-Cow / Hip Thrust for lower-back pain).
- ☑ `DietSuggestionsService` static map per condition — pattern (DASH / Mediterranean / low-GI / low-sodium), foods to favor, foods to limit, daily targets, rationale.
- ☑ **Doctor-disclaimer banner** on every condition-driven screen; 6-month re-confirm prompt via `lastDoctorReview` .

Diary, nutrition & activities

- ☑ **Food diary** — daily macro rings + 14-day history + custom-meal builder (iOS + Android Room).
- ☑ **Snap Meal (iOS)** — Vision / ML Kit on-device food classification → Open Food Facts macro lookup.
- ☑ **Read Label (iOS / Android)** — Vision / ML Kit text recognition → kcal / protein / carbs / fat regex extraction.
- ☑ **Barcode scanner** — VisionKit / ML Kit barcode → Open Food Facts → macros written to HealthKit as `HKCorrelation` .
- ☑ **Activities** — non-workout movement (walking, gardening, cycling, cleaning) with full add / edit / delete.

Medicine reminders

- ☑ iOS: `UNUserNotificationCenter` with notification category `MEDICINE_REMINDER` , Take / Snooze 10 min actions.
- ☑ Android: `AlarmManager.setRepeating` + `BroadcastReceiver` with same actions; `BootReceiver` re-arms after reboot.
- ☑ Persisted dose log + 14-day adherence streak + archive.
- ☑ Cron-like schedule (times of day × weekdays) stored as JSON for cross-platform export.
- ☑ Optional OpenFDA drug-info lookup via `/api/medicine/lookup` (no auth).

Health articles

- ☑ Bundled offline-readable `HealthArticleSeed` (works in guest mode) — 6 starter articles.
- ☑ Live MyHealthfinder topics via `/api/health/topics` (no auth).
- ☑ OpenFDA drug-label search via `/api/medicine/lookup` (no auth).

Apple Watch surfaces

- [x] `HKWorkoutSession` + `HKLiveWorkoutBuilder` with always-on display, HR zones, calories, elapsed time.
- [x] Run mode adds live pace.
- [x] GPS routes.
- [x] Workout Mirroring sender.
- [x] Live Activity (lock-screen).
- [x] Watch complication for Readiness on watch face.
- [x] iMessage app extension (`MyHealthMessages`) for activity cards.
- [x] Vertically-paged tabs: Quick Log → Live Workout → Run → **Anatomy** → Water → Weight → Mood → History → Settings.

Wear OS surfaces

- [x] Vertically-paged tabs (9 pages) mirroring the Apple Watch shape, incl. Anatomy.
- [x] Health Services for live HR + GPS via `ExerciseClient` .
- [x] **Tile** for Readiness.
- [x] **Complication** for Readiness.

Cross-platform sync & portability

- [x] Single canonical schema at `shared/schemas/myhealth.schema.json` — Apple Core Data and Android Room entities map 1:1.
- [x] iOS `PortabilityService.exportEverything()` writes a date-stamped JSON file to share via AirDrop / Files.
- [x] CloudKit sync for iOS / watchOS (`iCloud.com.fitfusion` , 15 entities) when user is signed in.
- [x] Erase-all-on-device action wipes every entity and resets onboarding.
- [ ] Android `data/portability/PortabilityService.kt` export (deferred — see DESIGN.md §4.11).

Backend API (Express + SQLite)

- [x] `GET /api/health-check` — liveness.
- [x] `POST /api/auth/register` , `POST /api/auth/login` — bcrypt + JWT.
- [x] `GET /api/profile` , `PUT /api/profile` — profile + BMI.
- [x] `POST /api/profile/metrics` , `GET /api/profile/metrics?type=…` — log + list.
- [x] `POST /api/nutrition/meal` , `GET /api/nutrition/today` — meals + totals.
- [x] `GET /api/insights/readiness` , `GET /api/insights/weekly` .
- [x] `GET /api/health/topics` , `GET /api/health/topic/:id` — MyHealthfinder (public).
- [x] `GET /api/health/drug?name=…` and `/api/medicine/lookup` — OpenFDA (public).

Settings

- [x] Account / sign in / guest toggle.

- [x] Imperial vs metric units.
- [x] Theme (Material 3 dynamic on Android; system / light / dark).
- [x] 5 localizations on Android (`en` , `es` , `fr` , `de` , `hi`).
- [x] Crash reports — Sentry (off by default).
- [x] Anonymous usage analytics — PostHog (off by default).
- [x] Export my data (JSON).
- [x] Erase all on-device data.
- [ ] Android Settings toggle for Health-conditions store (deferred — see WORKOUT-MEDIA.md §7).
- [ ] Android Settings "Sign in for cloud sync" wiring (deferred).

Telemetry & observability (all free tier, all opt-in)

- [x] **Sentry** — crashes + APM + logs (5K errors/mo): iOS · watchOS · Android · Wear · Expo · backend.
- [x] **PostHog** — product analytics + feature flags + replays (1M events/mo, OSS, EU region): iOS · Android · Expo · backend.
- [x] Privacy contract: every SDK off by default; wrappers strip `event.user` ; no health data, meal contents, medicine names, photos, or screen recordings ever sent.
- [x] Grafana Cloud Free + UptimeRobot — server metrics + uptime (docs only).

Privacy posture

- [x] All AI inference on-device (Core ML, Vision, ML Kit, BiologicalAgeEngine).
- [x] Nightly `MLUpdateTask` fine-tuning runs locally via `BGTaskScheduler` .
- [x] Guest Mode never sends personal data to any server.
- [x] Bundle ID `com.fitfusion.*` and CloudKit container `iCloud.com.fitfusion` retained so existing CloudKit data survives the rebrand.
- [x] HealthKit / Health Connect respect platform-level user permission gates.
- [x] Backend uses bcrypt-hashed passwords + JWT; tokens never leave AsyncStorage / UserDefaults / DataStore on the client.

Distribution

- [x] iOS App Store (`distribution/app-store/`) — privacy nutrition labels, what's-new copy.
- [x] Google Play (`distribution/play-store/`) — Data safety form, what's-new copy.
- [x] Desktop wrapper (Electron) under `desktop/` .
- [x] Marketing site at root (`index.html` , `styles.css` , `script.js`) — auto-deployed by `marketing.yml` .

NOT in v1 (explicitly)

- [ ] SensorKit deep-context (requires separate Apple entitlement approval).

- [ ] Apple Game Center leaderboards (`LeaderboardClient` flag, off by default).
- [ ] Apple Vision Pro spatial coach view.
- [ ] Offline workout playback (downloaded media).
- [ ] Insulin-dosing calculator, arrhythmia detection, ECG interpretation, calorie auto-meal-planner — all out of scope (FDA SaMD line).
- [ ] Android `PortabilityService` export.
- [ ] Android Settings toggle for Health-conditions opt-in.
- [ ] Wear OS Readiness tile real value (currently placeholder reading SharedPreferences).
- [ ] Release signing config for Android phone app (`signingConfigs.release`).

OBSERVABILITY

OBSERVABILITY.md

MyHealth — Observability & Analytics Stack

> **All free tier · all opt-in · all privacy-first**

A “full observability platform” usually means: **errors + logs + metrics + APM + replays + product analytics**. The big SaaS suites (Datadog, New Relic, Grafana Cloud) bundle these together but charge per host/user — out of reach for an early-stage app.

MyHealth uses a **pieced-together free stack** that gives you ~80% of the same capability for $0/month.

---

1. The chosen stack

Concern	Tool	Free tier	Why
Crash reporting	Sentry	5K errors/mo	Best-in-class stack-trace symbolication, 2024-grade SwiftPM/Gradle SDKs
Performance / APM	Sentry (same project)	included	We have `tracesSampleRate=0` by default; flip to 0.1 in prod when you want it
Session replay	Sentry Replay	50/mo	Disabled in MyHealth on privacy grounds — easy to flip on
Logs	Sentry Logs (2024 GA)	included	Structured logs in same dashboard as errors
Product analytics	PostHog	**1M events/mo**	Open source, GDPR-friendly, EU region available, **self-hostable** if you outgrow free
Feature flags	PostHog (same project)	included	Roll out new features by % of users
Heatmaps + funnels	PostHog (same project)	included	See where users drop off in onboarding
Server metrics	Grafana Cloud Free	10K series + 50 GB logs	Optional — only when the backend grows beyond a single VM
Uptime monitoring	UptimeRobot free	50 monitors, 5-min checks	Pings `/api/health-check`
Pages / status page	statuspage.io free	1 status page, 100 subs	Communicate incidents

Total cost: $0/month for any pre-MVP / MVP product.

---

2. Where MyHealth currently lives on the spectrum

	Datadog	New Relic	Grafana Cloud	MyHealth (Sentry + PostHog)
Errors / crashes	✅	✅	✅ (Faro)	✅ Sentry
Performance / APM	✅	✅	✅ (Tempo)	✅ Sentry tracing
Logs (centralized)	✅	✅	✅ (Loki)	✅ Sentry Logs
Infrastructure metrics	✅	✅	✅ (Prometheus)	⚪ Grafana Cloud Free (optional)
Uptime monitoring	✅	✅	–	⚪ UptimeRobot Free (optional)
Session replay	✅	✅	✅ (Faro)	✅ Sentry Replay (off by default)
Product analytics	–	–	–	✅ PostHog
Feature flags	–	–	–	✅ PostHog
Cohort funnels / heatmaps	–	–	–	✅ PostHog
Cost / month	$$$ \| $$ $	Free 10K	**$0**	

---

3. What's wired in this repo

iOS / watchOS

- `shared/FitFusionCore/Package.swift` declares `posthog-ios` + `sentry-cocoa` SwiftPM deps
- `CrashReportingService.swift` → Sentry wrapper (opt-in via Settings → "Send crash reports")
- `AnalyticsService.swift` → PostHog wrapper (opt-in via Settings → "Share anonymous usage analytics")
- Booted from `FitFusionApp.init()` only when toggles are on

Android phone (+ Wear)

- `android/app/build.gradle.kts` declares `io.sentry:sentry-android` + `com.posthog:posthog-android` deps
- `crash/CrashReportingService.kt` + `analytics/AnalyticsService.kt` mirror the iOS API
- Booted from `MyHealthApp.onCreate()` only when DataStore opt-in flags are true

- BuildConfig fields `SENTRY_DSN`, `POSTHOG_API_KEY`, `POSTHOG_HOST` injected from env vars at build time

Backend (Express)

- `server/middleware/sentry.js` + `server/middleware/analytics.js` are graceful no-op shims when env vars are unset
- `server.js` calls `Sentry.init` + `Analytics.init` early, then attaches Sentry request/error handlers around the routes
- npm deps: `@sentry/node` + `posthog-node`

Expo (React Native)

- `mobile/src/crash.js` (Sentry) + `mobile/src/analytics.js` (PostHog)
- Read DSN/key from `EXPO_PUBLIC_SENTRY_DSN` / `EXPO_PUBLIC_POSTHOG_API_KEY`
- Toggle stored under AsyncStorage key `crashReportsEnabled` / `analyticsEnabled`

---

4. Privacy contract — same on every platform

Field	Sentry sends?	PostHog sends?
Stack trace (file, line, fn)	✅	–
Device model + OS version	✅	✅
App version	✅	✅
Anonymous installation_id	✅	✅
Email / name / account ID	❌ never	❌ never
HealthKit / Health Connect data	❌ never	❌ never
Medicine names, doses, mood entries	❌ never	❌ never
Photos / screen recordings / videos	❌ never	❌ never (replay disabled)
URL query strings, POST bodies	❌ never	❌ never
Feature-use events (e.g. “meal_logged”)	–	✅ name only, no contents

Both wrappers strip `event.user` in `beforeSend` to belt-and-suspenders enforce zero-PII.

---

5. Event taxonomy (kept in sync across platforms)

```
onboarding_started              first launch screen rendered
onboarding_completed            user pressed "Enter MyHealth"
guest_mode_chosen               user tapped Continue as Guest
sign_in_completed               user signed in via the backend
workout_started                 timer began for any workout
workout_completed               timer ended (logged or aborted)
meal_logged                     a Meal entity was inserted
medicine_added                  a Medicine entity was inserted
medicine_dose_taken             user tapped Take on a reminder
bio_age_estimated               BiologicalAgeView was opened
data_export_triggered           Settings → Export pressed
data_erase_confirmed            Settings → Erase confirmed
```

Each of these is a single string defined in: - `shared/FitFusionCore/Sources/FitFusionCore/AnalyticsService.swift` (`AnalyticsEvent`) - `android/app/src/main/java/com/myhealth/app/analytics/AnalyticsService.kt` (`AnalyticsEvent`) - `mobile/src/analytics.js` (`Events`)

Keep these three lists in sync when adding a new event.

---

6. Mixpanel / Amplitude alternative

If you'd rather use Mixpanel or Amplitude instead of PostHog:

	Mixpanel	Amplitude	PostHog (default)
Free tier	20M events/mo	10M events/mo	**1M events/mo** + self-host
Open source	❌	❌	✅
GDPR-friendly defaults	⚠️	⚠️	✅
EU-region data residency	✅ paid	✅ paid	✅ free
Feature flags included	❌ paid	❌ paid	✅ free
Session replay included	❌ paid	❌ paid	✅ free (50/mo)
Self-host option	❌	❌	✅

To switch: replace the SDK call inside `AnalyticsService.{swift,kt,js}` with the Mixpanel/Amplitude equivalent. The public API (`track` , `identify` , `screen` , `reset`) is intentionally the same.

7. Optional: full Datadog/New Relic/Grafana setup

When the app + backend grow beyond single-host scale:

Component	Free option	Path
Server metrics + dashboards	**Grafana Cloud Free** (10K active series, 50 GB logs, 14-day retention)	Add `prom-client` to `server/`, expose `/metrics`; configure Grafana Cloud agent
Centralized logs	**Grafana Loki** in Grafana Cloud Free	Wire `winston` + `winston-loki` transport in backend
Distributed tracing	Sentry Tracing (free w/ existing project) OR Grafana Tempo Free	Already wired; flip `tracesSampleRate` from 0 → 0.1 in `CrashReportingService`
Real User Monitoring	**Sentry Browser/RN** (already wired) OR Grafana Faro Free	–
Synthetic monitoring	UptimeRobot Free (50 monitors, 5-min cadence)	Point at `/api/health-check`
Status page	statuspage.io free (1 page, 100 subs)	Embed iframe on marketing site

8. Setup checklist

1. **Sentry** — sign up at sentry.io/signup (free) → 5 projects → 5 DSN secrets in GitHub. See `SENTRY.md`.
2. **PostHog** — sign up at posthog.com/signup (free) → 1 project → 1 API key (used for both phone + watch since it's the same product). Add 5 secrets:
   - `POSTHOG_API_KEY_IOS`, `POSTHOG_API_KEY_ANDROID`, `POSTHOG_API_KEY_SERVER`, `POSTHOG_API_KEY_EXPO` (or share one key — both works)
   - `POSTHOG_HOST` (defaults to `https://eu.i.posthog.com`)
3. **UptimeRobot** (optional) — sign up free → add HTTP monitor on `https://api.myhealth.app/api/health-check`.
4. **Grafana Cloud Free** (only when needed) — sign up free → install agent on backend host → grab dashboards from grafana.com.

Until those secrets are wired, every wrapper in this repo is a **silent no-op** — nothing breaks, nothing leaks.

9. Honest limitations of this stack

- **No infrastructure-level monitoring** (CPU, memory, disk, network) without adding Grafana Cloud or Datadog. Sentry/PostHog see only your app's perspective.
- **No SLO tracking** — for that you'd want Grafana SLO or Sentry's paid SLO add-on
- **No PagerDuty / on-call rotation** integration — use Sentry's free Slack alert action and a free PagerDuty Developer plan
- Free tiers are **soft caps** — exceeding them means you stop getting new events, you don't get billed

For an early-stage app: this is more than enough. For a 100k MAU app with a paying SLA: graduate to Grafana Cloud Pro ($49/mo) + Sentry Team ($26/mo) + PostHog $0 (still free up to 1M events).

PRIVACY

PRIVACY.md

MyHealth — Privacy Policy

Last updated: 2026-05-06

TL;DR

- **Optional account.** "Guest Mode" works without login on every platform. If you create an account, it lives on the MyHealth backend you (or your fork) control.
- **No tracking by default.** Crash reporting (Sentry) and product analytics (PostHog) ship as *off-by-default* and only run if you opt in via Settings → Privacy.
- **Health data is yours.** Apple HealthKit (iPhone, Apple Watch) and Android Health Connect (Android phone, Wear OS) are the source of truth. We never copy HealthKit/Health Connect data to our backend, never log it to Sentry, and never send it to PostHog.
- **Open source.** Every line is at https://github.com/AmericanGroupLLC/HealthApp under the MIT License. Audit anything; build it yourself if you don't trust us.

What MyHealth stores

On your device, in OS-private storage that's wiped if you uninstall:

- Your privacy preferences (which opt-in toggles you've enabled).
- Your last-viewed dashboard tab, theme, and unit preferences (kg/lb, ml/oz).
- Your local workout history (CoreData on iOS, Room on Android, Expo SQLite on the React Native build).
- A cache of media (workout GIFs, exercise illustrations) downloaded for offline use.
- Your AI coach conversation history (on-device only — the local LLM never leaves the device).

In Apple HealthKit / Android Health Connect (you control via OS settings):

- Workouts, heart rate, steps, weight, water, mindfulness sessions, state-of-mind, sleep stages.
- We **read** these to show you trends and **write** new entries (e.g., a workout you finished). We never read more than is needed for the screen you're on. Permissions are granular: you can grant read-only, write-only, or both per data type.

On the MyHealth backend (only if you create an account):

- Email + bcrypt-hashed password (or OAuth provider claim, if signed in via Apple/Google).
- Workout templates and challenges you create or join.
- Account-level preferences (display name, avatar URL, time zone).
- We **do not** store HealthKit/Health Connect data server-side. Workouts synced to the backend are summary records (`type` , `duration` , `kcal` , `start_at`); full sample data stays in HealthKit/Health Connect.

What MyHealth sends off-device (only if you opt in)

Crash reports — Sentry (off by default)

Field	Sent?
OS version, device model, app version	✅ yes
Stack trace, breadcrumbs (last ~30 user actions, sanitized)	✅ yes
Anonymous installation ID	✅ yes
User email, account ID	❌ never — wrapper strips `event.user`
HealthKit / Health Connect data	❌ never — never logged
Meal contents, medicine names, photos, screen recordings	❌ never
AI coach conversation history	❌ never

Sentry is wired into iOS, watchOS, Android, Wear OS, the Expo build, and the backend. See `SENTRY.md` for the wrapper code and the exact DSN-resolution order.

Product analytics — PostHog (off by default)

We capture **screen views**, **feature taps** (e.g., "started workout"), and **funnel events** (e.g., "completed onboarding"). We capture event names and non-PII properties. We **never** capture: workout data values, meal contents, medicine names, biometrics, HealthKit/Health Connect data, photos, or text you type into the AI coach.

PostHog is self-hostable (EU region by default for the OSS option). See `OBSERVABILITY.md` for the full pipeline.

Backend — your data, on a server you (or we) operate

When you sign in, your account credentials and the summary records described above transit over HTTPS to the MyHealth backend. The backend itself is open-source (see `server/`) and self-hostable.

What MyHealth never sends, ever

- Your full HealthKit / Health Connect sample data.
- Photos taken in-app (form-check videos for AI analysis are processed on-device).
- AI coach conversation contents.
- Medicine names, dosages, or schedules.
- Meal photos or food log contents.

Third-party SDKs

SDK	Purpose	Default	Where it runs
`getsentry/sentry-cocoa`	Crash reporting (iOS, watchOS)	OFF	Device
`io.sentry:sentry-android`	Crash reporting (Android, Wear)	OFF	Device
`@sentry/node`	Crash reporting (backend)	OFF	Server
`@sentry/react-native`	Crash reporting (Expo build)	OFF	Device
`posthog-ios`	Product analytics (iOS)	OFF	Device
`posthog-android`	Product analytics (Android)	OFF	Device
`posthog-node`	Product analytics (backend)	OFF	Server
`posthog-react-native`	Product analytics (Expo)	OFF	Device
Apple HealthKit	Health data read/write	(user grants per-type)	Device only
Android Health Connect	Health data read/write	(user grants per-type)	Device only
Apple WorkoutKit	Workout suggestions / scheduling	OS-managed	Device
Apple SignIn / Google SignIn	Optional account creation	(user-initiated)	Device → backend
On-device LLM (`llama.cpp` family)	AI coach	(user-initiated)	Device only

Marketing site

The marketing site at https://americangroupllc.github.io/HealthApp/ uses **Google Analytics (GA4)** for visitor counts on the public landing page only. This is independent of the app itself; once you install the app, GA4 is not involved. Marketing-site analytics can be disabled by browser-level do-not-track preferences and ad blockers.

Children

MyHealth is rated 12+ in App Store and “Teen” in Play. Parts of the AI coach discuss exercise intensity and nutrition that may not be appropriate for younger users. We do not knowingly collect data from children under 13 (or the local minimum age, where higher).

Your rights

- **Access** — request an export of your account data via mailto:privacy@americangroupllc.com. We respond within 30 days.
- **Delete** — delete your account from Settings → Account → Delete Account, or email the address above. Local on-device data is wiped on uninstall.
- **Withdraw consent** — turn off Sentry / PostHog at any time in Settings → Privacy. Past events that were sent before you opted out are not retroactively deleted from Sentry/PostHog (we’ll delete the entire project on request).
- **Data Subject Requests (GDPR/CCPA)** — same email; same 30-day SLA.

Changes to this policy

We’ll bump *Last updated* at the top and post a brief note in the in-app What’s New on any change that affects what we collect or how. Material changes will require fresh consent.

Contact

privacy@americangroupllc.com

PRODUCTION

PRODUCTION.md

MyHealth — Production Readiness & Store Deployment

> **Honest answer**: the **CI/CD pipeline is production-grade**; the **app code itself needs ~3 weeks of polish + the unavoidable store fees** before it's accepted by Apple App Review or Google Play Review.
>
> Everything below is **free** except the two store fees.

---

1. Are you ready to ship today? — gap audit

Status	Item	What's needed
✅ Done	iOS app builds, signs, and uploads via CI	–
✅ Done	Android phone APK + AAB builds and uploads via CI	–
✅ Done	Wear OS APK builds	–
✅ Done	iOS Simulator UI tests + Android unit tests	–
✅ Done	Backend smoke + Jest tests pass	–
✅ Done	GitHub Release artefact pipeline	–
✅ Done	Play Store auto-upload (`r0adkll/upload-google-play`)	Just needs the Service Account secret
✅ Done	TestFlight auto-upload (`Apple-Actions/upload-testflight-build`)	**Just added in this commit** — needs App Store Connect API key secret
✅ Done	Onboarding · Guest Mode · 5-tab nav · Vitals · Bio Age · Medicine reminders · Diary	–
⚠️ Gap	**Apple Developer Program membership**	**$99 / year** — unavoidable. Without it Xcode won't even sign for device.
⚠️ Gap	**Google Play Developer account**	**$25 one-time** — unavoidable. Without it, Play Console rejects your AAB.
⚠️ Gap	**Real app icon (1024 × 1024 PNG)**	5 min in any free icon tool (icon.kitchen is free)
⚠️ Gap	**App Store screenshots** (iPhone 6.7″ + 5.5″, iPad 12.9″, Apple Watch)	Generated via `fastlane snapshot` — free
⚠️ Gap	**Play Store screenshots** (Phone, 7″ tablet, 10″ tablet, Wear)	Generated via `fastlane screengrab` — free
⚠️ Gap	**Privacy policy URL**	Required by both stores. Use GitHub Pages site — free.
⚠️ Gap	**App description, keywords, category**	Manual entry in Play Console + App Store Connect, ~30 min

Status	Item	What's needed
⚠️ Gap	**Apple Health usage descriptions** in `Info.plist`	Already present (`NSHealthShareUsageDescription` etc.) — review wording
⚠️ Gap	**HealthKit privacy declaration** in App Store Connect	Manual form, ~15 min
⚠️ Gap	**Play Data Safety form**	Manual form in Play Console, ~30 min
⚠️ Gap	**Real `.mlmodel` files** for AdaptivePlanner + FoodClassifier	App falls back to heuristic; OK to ship without
⚠️ Gap	**Real exercise demo videos**	Detail page falls back to placeholder; OK to ship
⚠️ Gap	**Sign in with Apple** (App Store rule 4.8)	Only required if you ship login at all. Guest Mode means we may be exempt — verify with App Review.
⚠️ Gap	**Account deletion endpoint** (App Store rule 5.1.1(v) / Play Data Deletion)	Wire `DELETE /api/auth/me` in backend; surface "Delete account" in Settings. ~2 hours.
⚠️ Gap	**Real app icon assets** in iOS Assets.xcassets / Android mipmap	Drop a 1024×1024 PNG; everything else is templated
⚠️ Gap	**Production-grade signing keys** (release keystore for Android, distribution provisioning for iOS)	Generate once, store as Base64 GitHub Secret — see below
⚠️ Optional	**Crash reporting** (Sentry / Firebase Crashlytics)	Both have free tiers. Recommended before public beta.
⚠️ Optional	**Analytics** (PostHog / Plausible / Umami self-hosted)	Free open-source options
🔵 Future	Apple Health entitlement approval	Auto-approved for HKHealthStore reads; some types need explicit declaration
🔵 Future	"Made for kids" / age gating	Not applicable unless you target under-13 users

Status	Item	What's needed
🔵 Future	EU AI Act / FDA clearance	Only if you make diagnostic claims (we explicitly don't — bio-age has disclaimers)

Effective time to first store submission: ~3 weeks of polish + waiting for Apple Review (1-3 days first time) + Play Review (a few hours typically).

---

2. Free deploy path — who pays for what

Component	Cost	Why
GitHub Actions (public repo)	**FREE — unlimited minutes**	Public repos get unlimited Linux + macOS minutes
GitHub Actions (private repo)	2,000 free min/mo Linux, **200 free min/mo macOS** (10× billing multiplier)	Should be enough for ~10 releases/month
Fastlane	**FREE (MIT)**	OSS by Felix Krause
`r0adkll/upload-google-play`	**FREE (MIT)**	OSS GitHub Action
`Apple-Actions/upload-testflight-build`	**FREE (MIT)**	Maintained by Apple
Codecov (public repo)	**FREE — unlimited**	Free tier covers public repos
GitHub Pages (privacy policy site)	**FREE — unlimited**	Static hosting
Apple Developer Program	**$99 / year**	**Unavoidable** — Apple's gate
Google Play Developer	**$25 one-time**	**Unavoidable** — Google's gate

Total cash to ship: ~$124 first year, $99/year after.

---

3. The wired-up CI/CD path (what you already have)

```
git tag v1.0.0
git push origin v1.0.0
        ↓
.github/workflows/release.yml fires
        ↓
        ├─ build-android       (ubuntu, free)        →  APK + AAB
        ├─ build-ios           (macos, ~5-10 min)    →  unsigned XCArchive
        ├─ build-server        (ubuntu, free)        →  tarball
        ├─ build-web           (ubuntu, free)        →  zip
        ↓
publish-github-release   →  Creates Release at v1.0.0 with all artefacts
        ↓
publish-play-store       →  Uploads AAB to Play Console (gated by secret)
publish-testflight       →  Builds signed .ipa, uploads to TestFlight
                              (gated by App Store Connect API key secret)
```

Both store-upload jobs are **conditionally skipped** if the corresponding secret isn't set, so the workflow runs cleanly even before you've configured deployment credentials.

---

4. One-time setup checklist

Android → Google Play

1. Pay **$25** at https://play.google.com/console/signup
2. Create a new app for `com.myhealth.app`
3. Settings → API access → **Create new service account** in Google Cloud Console
4. Grant the service account "Release Manager" role on your app
5. Download the service-account JSON key
6. **Manually upload one APK** to Play Console once (Google requires this so it can register the package name)
7. Generate an upload keystore:

   ```
   keytool -genkey -v -keystore upload.jks -keyalg RSA -keysize 2048 -validity
           10000 -alias upload
   ```

8. In GitHub repo → Settings → Secrets → New repository secret (one each):
   - `PLAY_STORE_SERVICE_ACCOUNT_JSON` — paste the raw JSON
   - `PLAY_STORE_PACKAGE_NAME` — `com.myhealth.app`

    - `ANDROID_KEYSTORE_BASE64` — `base64 -i upload.jks | pbcopy`
    - `ANDROID_KEYSTORE_PASSWORD` — keystore password
    - `ANDROID_KEY_ALIAS` — `upload`
    - `ANDROID_KEY_PASSWORD` — key password

9. **Done.** Next `git tag v*` push uploads automatically.

iOS → TestFlight + App Store

1. Pay **$99/year** at https://developer.apple.com/programs/enroll/
2. In Xcode (one time): create the App ID `com.fitfusion.ios` + bundle IDs for the 3 extensions
3. App Store Connect → Users and Access → **Keys** → Generate API key with "App Manager" role
4. Download the `.p8` private key file
5. Note the **Key ID** and **Issuer ID** (visible on the Keys page)
6. In GitHub repo → Settings → Secrets:
    - `APP_STORE_CONNECT_API_KEY_ID` — e.g. `XYZ1234567`
    - `APP_STORE_CONNECT_API_ISSUER_ID` — UUID
    - `APP_STORE_CONNECT_API_KEY_P8_BASE64` — `base64 -i AuthKey_XYZ.p8 | pbcopy`
    - `APPLE_TEAM_ID` — 10-char team ID from Membership page
7. **Optional but recommended** — Fastlane Match for shared code-signing:
    - Create a private GitHub repo (e.g. `MyHealth-Certs`)
    - `fastlane match init` → choose `git` storage → point at that repo
    - `fastlane match appstore` once locally to populate it
    - Add secrets `MATCH_GIT_URL` (e.g. `git@github.com:you/MyHealth-Certs.git`) and `MATCH_PASSWORD` (the symmetric encryption key Match prompted you for)
    - The CI job will read-only sync the cert/profile from this repo
8. **Done.** Next `git tag v*` push builds a signed `.ipa` and uploads to TestFlight automatically.

---

5. Three-week pre-launch polish checklist

Week	What to do
Week 1 — Visuals + Privacy	• Make a 1024×1024 app icon and drop into `ios/FitFusion/Assets.xcassets/AppIcon.appiconset` and `android/app/src/main/res/mipmap-anydpi-v26/` • Take 6 screenshots per platform/device class (`fastlane snapshot` / `screengrab`) • Write a 1-page privacy policy and host it on GitHub Pages • Write the app description, short tagline, keywords (App Store) / category, contact email
Week 2 — Compliance	• Add **Delete Account** button in Settings + `DELETE /api/auth/me` route (App Store 5.1.1(v) + Play Data Deletion) • Fill the Apple Privacy Nutrition Labels (App Store Connect → App Privacy) • Fill Play Data Safety form • Verify all `Info.plist` `NSHealthShareUsageDescription` strings are clear + honest • Add Sentry / Crashlytics free tier for crash reports
Week 3 — Beta + dogfood	• Tag `v0.9.0-beta1` → push → automatic upload to TestFlight (internal) + Play **internal** track • Recruit 5-20 testers via TestFlight + Play Internal email lists • Iterate on feedback; fix bugs • When ready: bump to `v1.0.0`, push, and submit for review (no code change required — the workflow just promotes to `production` track based on the tag suffix)

---

6. Things you do NOT need to worry about

- ✅ The build pipeline (already wired)
- ✅ Code signing automation (Match + ASC API key)
- ✅ The version-bump script (`scripts/bump-version.sh`)
- ✅ Release notes (`distribution/whatsnew/whatsnew-en-US/whatsnew.txt` already templated)
- ✅ macOS runner cost (your repo is public → free)

---

7. The realistic answer to “is it production-ready?”

Question	Answer
Will it pass App Review **today**?	❌ Probably no — needs the icon, screenshots, privacy policy, account-deletion route, privacy form
Will it crash and burn on a real device?	❌ Probably no — but you should TestFlight + Play Internal-track for ~2 weeks first to catch device-specific issues
Is the architecture production-quality?	✅ Yes — privacy-first, local-first, on-device AI, multi-platform, well-tested CI
Can you deploy a beta this week?	✅ Yes — set up the 8 secrets above, tag `v0.1.0-beta1`, push. The CI does the rest.
Can you deploy to **production** this week?	⚠️ Technically yes (Apple/Google won't *prevent* it), but not recommended without the 3-week polish above

Bottom line: the **path** to production is fully wired and free; the **content** still needs ~3 weeks of human polish + $124 in store fees.

QUICKSTART

QUICKSTART.md

MyHealth — 5-Minute Quick-Start

Each platform fits on one screen. Pick what you have a machine for.

Platform	OS you need	Time
🌐 **Marketing site**	any	30 sec
🔌 **Backend (Express + SQLite)**	macOS / Linux / Windows	3 min
📱 **Expo (Android + iOS + Web)**	macOS / Linux / Windows	5 min
🍎 **iOS app**	macOS only	5 min
⌚ **watchOS app**	macOS only	5 min
📲 **Android phone app**	macOS / Linux / Windows	5 min
⌚ **Wear OS app**	macOS / Linux / Windows	5 min

For deeper testing scenarios see `TESTING.md`. For releases see `RELEASING.md`.

🌐 Marketing site — 30 seconds

1. **Open** `index.html` in any browser.

That's it. No build step, no server. Tabs, gradients, smooth scrolling all work standalone.

🔌 Backend (Express + SQLite) — 3 minutes

Prereqs: Node.js ≥ 18.

```
cd server
cp .env.example .env
## Edit .env: set JWT_SECRET to any 32-char random string.
## On Windows over a network share, point DB_PATH to a local temp dir
## (e.g. C:\Users\you\AppData\Local\Temp\myhealth.db) — SQLite WAL
## doesn't work on SMB shares.
npm install
npm run dev      # → ✅ MyHealth API listening on http://localhost:4000
```

Verify:

```
## In a second terminal:
cd server && npm run smoke
## Expected: 13 / 13 passed.
```

Troubleshooting: `database is locked` → move `DB_PATH` to a local-disk path.

📱 Expo (cross-platform RN) — 5 minutes

Prereqs: Node.js ≥ 18 + the Expo Go app on your phone.

```
cd mobile
npm install
npm start
```

Then: - Press `w` for **web** - Press `i` for **iOS simulator** (macOS only) - Press `a` for **Android emulator** - Or **scan the QR** with Expo Go on a real Android/iOS phone.

First screen = Login → tap **Continue as Guest** to skip account creation entirely.

Troubleshooting: blank QR → `npx expo start --tunnel` (works through restrictive networks).

🍎 iOS app — 5 minutes (macOS only)

Prereqs: macOS · Xcode 15+ · Homebrew. Free Apple ID is fine.

```
brew install xcodegen
cd ios
xcodegen generate
open FitFusion.xcodeproj
```

In Xcode: 1. Edit `ios/project.yml` → set `DEVELOPMENT_TEAM` to your 10-char Apple Team ID (or leave blank for simulator only). 2. Pick scheme **FitFusion** + an **iPhone 15** simulator. 3. ▶️ Run.

First screen = Login → **Continue as Guest** → 4-page Onboarding → Home.

Troubleshooting: "No such module 'FitFusionCore'" → Xcode → File → Packages → Reset Package Caches.

For one-line build-+-launch:

```
./scripts/run-ios-sim.sh
```

⌚ watchOS app — 5 minutes (macOS only)

Prereqs: same as iOS (Xcode + xcodegen).

```
cd watch
xcodegen generate
open HealthAppWatch.xcodeproj
```

In Xcode: 1. Pick scheme **HealthAppWatch** + an **Apple Watch Series 10 (46mm)** simulator (watchOS 11+). 2. ▶️ Run.

Pages (swipe up): Quick Log → Live Workout → Run → **Anatomy** → Water → Weight → Mood → History → Settings.

For real-device testing pair an Apple Watch via the iOS Watch app, set the same `DEVELOPMENT_TEAM` in `watch/project.yml`, and run on device.

🤖 Android phone app — 5 minutes

Prereqs: JDK 17 + Android Studio Hedgehog or newer (or just `cmdline-tools` + an SDK).

```
cd android
## First time only — generates the gradle wrapper:
gradle wrapper --gradle-version 8.10
chmod +x gradlew

./gradlew :app:installDebug   # installs on connected emulator/device
```

Or open `android/` in **Android Studio** → "Sync Project" → ▶️ on a Pixel emulator.

First screen = 4-page Onboarding (no login) → Bottom nav: Home · Train · Diary · Sleep · More.

Troubleshooting: - `SDK location not found` → set `local.properties` with `sdk.dir=/path/to/Android/sdk`. - Health Connect permission missing → install "Health Connect by Android" from Play Store on the emulator first.

For one-line boot-emulator-+-launch:

```
ANDROID_HOME=/path/to/sdk ./scripts/run-android-emulator.sh
```

⌚ Wear OS app — 5 minutes

Prereqs: same as Android phone + a Wear OS AVD created in Android Studio.

```
cd android
./gradlew :wear:installDebug
```

Pages (swipe up): Quick Log → Live Workout → Run → **Anatomy** (drills into shared `core` exercise library) → Water → Weight → Mood → History → Settings.

Watch face → long-press → **Add complication** → MyHealth Readiness.

For one-line:

```
WEAR_AVD_NAME=Wear_Round_API_33 ./scripts/run-wear-emulator.sh
```

🧪 Run every test on your machine — 1 line

```
./scripts/test-all.sh
```

Runs every suite available on the current OS: - Backend Jest + smoke (always) - Android `:core:test*` + `:app:test*` (if Gradle installed) - Swift Package tests + iOS sim build (if on macOS with Xcode) - Marketing site lint (if `npx` available)

Skips suites that need tooling you don't have.

🚀 Cut a release — 2 lines

```
./scripts/bump-version.sh 1.2.0
git commit -am "chore(release): v1.2.0" && git tag v1.2.0 && git push origin main
        v1.2.0
```

GitHub Actions does the rest: - Builds APK + AAB + Wear APK + iOS xcarchive + server tarball + web zip - Creates a **GitHub Release** at `v1.2.0` with auto-changelog and all artefacts - Optionally uploads the AAB to **Google Play Store** if `PLAY_STORE_SERVICE_ACCOUNT_JSON` secret is set

See `RELEASING.md` for the full setup.

🆘 Help

- **Architecture & data flow**: `DESIGN.md`
- **Sanity-test walkthroughs per feature**: `TESTING.md`
- **Releases + Play Store + App Store**: `RELEASING.md`
- **iOS code-signing**: `ios/SIGNING.md`
- **Drop-in `.mlmodel` files**: `ios/FitFusion/Models/README.md`
- **Cross-platform JSON schema**: `shared/schemas/README.md`

RELEASE-VIDEO-SCRIPT

RELEASE-VIDEO-SCRIPT.md

Release Video Script — FitFusion

This file drives the silent MP4 release video built by the umbrella's `release-video.yml` reusable workflow at tag time.

Each `## Heading | <duration_seconds>` defines one scene. Within the scene, list keys as bullets. Recognised keys:

- `viewport: WIDTHxHEIGHT` (default 1280x720)
- `scroll: <px|%|top|bottom>` (default 0)
- `wait: <seconds|ms>` (dwell time after navigation)
- `caption: <text>` (placeholders `{APP_NAME}` and `{VERSION}`)
- `duration: <seconds>` (override the heading-level duration)

The intro and outro cards (3 s each) are auto-generated from the brand colour in `release.config.json`.

> **Tagline:** Workouts, vitals, and recovery in one place.

Hero — top of marketing site | 4

- viewport: 1280x720
- scroll: top
- wait: 600ms
- caption: {APP_NAME} {VERSION}

Features overview | 5

- viewport: 1280x720
- scroll: 25%
- wait: 600ms
- caption: New in {VERSION}

Highlight reel | 5

- viewport: 1280x720
- scroll: 50%
- wait: 600ms
- caption: Built for everyone

What's next | 5

- viewport: 1280x720
- scroll: 75%
- wait: 600ms
- caption: Available everywhere

Call to action — bottom of page | 4

- viewport: 1280x720
- scroll: bottom
- wait: 800ms
- caption: Try {APP_NAME} today

RELEASING

RELEASING.md

MyHealth — Releasing & Distribution

This document covers **how to cut a release**, **what GitHub Actions does automatically**, and **whether direct Play Store / App Store deployment is possible** (spoiler: yes for Play, conditional for App Store).

---

1. TL;DR — Cut a release

```
## 1. Pick a version number that follows semver
git tag v1.2.0
git push origin v1.2.0
```

That's it. The `release.yml` workflow under `.github/workflows/` will automatically:

1. Build the **Android phone APK** (`MyHealth-v1.2.0.apk`)
2. Build the **Wear OS APK** (`MyHealth-Wear-v1.2.0.apk`)
3. Build the **server tarball** (`myhealth-server-v1.2.0.tgz`)
4. Build the **marketing-site zip** (`myhealth-web-v1.2.0.zip`)
5. Upload **iOS XCArchive** + **iOS .ipa** when run on a macOS runner (skipped on Linux runners — no Xcode)
6. Create a **GitHub Release** at `v1.2.0` with:
   - Title: "MyHealth v1.2.0"
   - Body: auto-generated changelog from commit messages since the previous tag
   - All built artefacts attached as downloads
7. Apply tag labels (`release` , `android` , `ios` , `wear`) for filtering

If the tag uses `v*-rc*` (e.g. `v1.2.0-rc1`) the release is marked **pre-release**.

---

2. Workflow files

File	Trigger	What it does
`.github/workflows/ci.yml`	every push + PR	Backend tests · Android build · Compose lint · iOS Swift Package tests (macOS runner)
`.github/workflows/release.yml`	push of `v*` tag	Build all binaries, attach to GitHub Release, optionally upload to Play Store
`.github/workflows/android.yml`	push to main + PR	Android-specific: lint, unit, instrumented (when emulator runner available)
`.github/workflows/ios.yml`	push to main + PR	iOS Swift Package tests + watch tests (macOS runner)
`.github/workflows/backend.yml`	push touching `server/**`	Backend Jest + smoke run
`.github/workflows/marketing.yml`	push touching `index.html` / `styles.css` / `script.js`	Validates HTML/CSS, optionally publishes to GitHub Pages

All workflows are committed under `.github/workflows/`.

---

3. Direct Google Play Store upload — yes, fully automated

Yes — **the release workflow can publish directly to Google Play** without any manual step in the Play Console. The mechanism:

1. **Create a Google Play Service Account** in Play Console → Setup → API access.
2. Grant it permission to **Release manager** for the `com.myhealth.app` package.
3. Download the resulting JSON key.
4. Add it as a GitHub secret:
   - `PLAY_STORE_SERVICE_ACCOUNT_JSON` — raw contents of the JSON file
   - `PLAY_STORE_PACKAGE_NAME` — e.g. `com.myhealth.app`

5. The release workflow uses `r0adkll/upload-google-play@v1` to push the AAB to a track of your choice.

In `release.yml` the publish step is **gated by an env flag** so you can land the workflow before Play Console setup:

```
- name: Publish to Google Play (internal track)
  if: env.PLAY_STORE_SERVICE_ACCOUNT_JSON ≠ ''
  uses: r0adkll/upload-google-play@v1
  with:
    serviceAccountJsonPlainText: ${{ secrets.PLAY_STORE_SERVICE_ACCOUNT_JSON }}
    packageName: com.myhealth.app
    releaseFiles: android/app/build/outputs/bundle/release/app-release.aab
    track: internal     # internal / alpha / beta / production
    status: completed
    inAppUpdatePriority: 2
    whatsNewDirectory: distribution/whatsnew
```

Tracks supported: `internal`, `alpha`, `beta`, `production`. Most teams promote through them: `internal` → `alpha` → `beta` → `production`.

For the first publish, you must **manually upload an APK once** to Play Console so it can register the package name. After that, every subsequent release flows through CI.

What's-new release notes

Drop short release notes in `distribution/whatsnew/whatsnewen-US/whatsnew.txt` (and other locales). The workflow reads `whatsnewDirectory` and uploads them to the matching Play Console fields.

---

4. Apple App Store — partial automation

Apple is **stricter** than Google, but still scriptable:

- **TestFlight upload** — fully automatable via `fastlane pilot` or `xcrun altool`. Requires an App-Specific Password or App Store Connect API key (a `.p8` file). See `ios/SIGNING.md` for setup.
- **Production release** — App Review remains a **manual + opaque** human review step. Once approved, you can press "Release Now" via App Store Connect API or trigger the release manually.

The `release.yml` workflow gates the iOS upload behind macOS runners and the presence of `APP_STORE_CONNECT_API_KEY` secret — same pattern as Play.

---

5. Wear OS distribution

Wear OS apps ship through the **same Play Console package**. The release workflow builds `:wear:assembleRelease` alongside `:app:assembleRelease` and the **Play Store automatically associates the Wear APK** with the phone APK (both share `com.myhealth.app` family of bundle IDs and the workflow zips them into a single AAB).

For standalone Wear OS distribution, set `com.google.android.wearable.standalone = true` in the Wear manifest (already done) and submit through the **Wear OS** track in Play Console.

---

6. Versioning

Use **SemVer**:

Tag pattern	Meaning	Play track default
`v1.0.0`	Stable	`production` (gated — flip the env flag)
`v1.1.0`	Minor (new features, backwards-compat)	`beta`
`v1.0.1`	Patch (bug fixes)	`production`
`v1.2.0-rc1`	Release candidate	`alpha`
`v1.2.0-beta1`	Public beta	`beta`
`v1.2.0-internal1`	Internal QA	`internal`

Bump these matching values **in lockstep** before tagging:

File	Field
`android/app/build.gradle.kts`	`versionName` + `versionCode`
`android/wear/build.gradle.kts`	`versionName` + `versionCode`
`ios/FitFusion/Info.plist`	`CFBundleShortVersionString` + `CFBundleVersion`
`watch/HealthAppWatch/Info.plist`	same
`mobile/app.json`	`expo.version` + `android.versionCode` + `ios.buildNumber`
`server/package.json`	`version`

Or run the helper:

```
./scripts/bump-version.sh 1.2.0
```

---

7. Distribution checklist

Before pushing the tag:

- [ ] Bumped versions in all manifests (script above)
- [ ] Updated `RELEASE_NOTES.md` with the highlight reel
- [ ] Updated `distribution/whatsnew/whatsnew-en-US/whatsnew.txt` (≤ 500 chars per Play)
- [ ] Ran `./scripts/test-all.sh` locally — all green
- [ ] Squashed the diff and merged to `main`
- [ ] Signed the Android AAB (configured via `keystore.properties` — never committed)
- [ ] Confirmed iOS provisioning profile is current
- [ ] Tagged with `git tag vX.Y.Z && git push origin vX.Y.Z`

After CI completes:

- [ ] Verify GitHub Release contains all expected artefacts
- [ ] Verify Play Console shows the new bundle on the chosen track
- [ ] Smoke test the downloaded APK on a real Android device
- [ ] Promote to next track in Play Console when ready

---

8. First-time secrets setup

In repo Settings → Secrets and variables → Actions → New repository secret, add:

Secret	What it is
`PLAY_STORE_SERVICE_ACCOUNT_JSON`	Play Service Account JSON (raw)
`PLAY_STORE_PACKAGE_NAME`	e.g. `com.myhealth.app`
`ANDROID_KEYSTORE_BASE64`	Base64 of the upload keystore
`ANDROID_KEYSTORE_PASSWORD`	Keystore password
`ANDROID_KEY_ALIAS`	Key alias inside the keystore
`ANDROID_KEY_PASSWORD`	Per-key password
`APP_STORE_CONNECT_API_KEY_ID`	(iOS) From App Store Connect
`APP_STORE_CONNECT_API_ISSUER_ID`	(iOS)
`APP_STORE_CONNECT_API_KEY_P8_BASE64`	(iOS) Base64 of the .p8 file
`APPLE_TEAM_ID`	(iOS) e.g. ABCD123456

Without these, the workflow still runs but **skips the upload steps** — artefacts are still attached to the GitHub Release for manual download.

---

9. Local pre-release dry run

```
./scripts/release-dry-run.sh v1.2.0
```

This builds every artefact locally without pushing anything — useful to catch issues before the tag is in.

SENTRY

SENTRY.md

MyHealth — Sentry Crash Reporting

> **Free Developer tier · Opt-in · Privacy-first**

Sentry is wired into every platform but **off by default**. The user must explicitly turn on “Send crash reports” in Settings, AND a DSN must be configured at build time, before a single byte leaves the device.

---

1. Free tier reality check

Plan	Cost	Errors/month	Performance events	Replays	Retention
Developer	**$0**	5,000	10,000	50	30 days
Team	$26/mo	50,000	100,000	500	90 days

For early-stage apps the **free Developer tier is more than enough** — 5,000 errors/month covers thousands of MAUs at typical crash rates. Sign up at sentry.io/signup.

We deliberately **disable performance tracing** (`tracesSampleRate: 0`) on every platform so quota is reserved for actual crashes.

---

2. Privacy posture (what gets sent and what doesn’t)

Field	Sent?
Stack trace (file, line, function)	✅ yes
Device model + OS version	✅ yes
App version + build	✅ yes
Anonymous device ID (Sentry's `installation_id`)	✅ yes
Email, name, account id	❌ never — `sendDefaultPii: false` + `beforeSend` strips `event.user`
HealthKit / Health Connect data	❌ never — never logged
Medicine names, doses, mood entries	❌ never — never logged
Photos, screen recordings, videos	❌ never — replays disabled
URL query strings or POST bodies	❌ never — request bodies not captured

The `beforeSend` callback on every platform strips `event.user` to belt-and- suspenders enforce zero-PII even if a future SDK update tries to attach it.

---

3. Wiring per platform

iOS / watchOS

- **Dependency**: `getsentry/sentry-cocoa` (SwiftPM, in `Package.swift`)
- **Wrapper**: `shared/FitFusionCore/Sources/FitFusionCore/CrashReportingService.swift`
- **Boot**: `FitFusionApp.init` calls `CrashReportingService.shared.bootstrapIfEnabled(release:)`
- **DSN source** (in priority order):
  1. Env var `SENTRY_DSN` (CI / dev)
  2. `Info.plist` key `SentryDSN` (release builds; injected by xcodegen build setting)
- **User toggle**: Settings → Privacy → "Send crash reports"

Android phone

- **Dependency**: `io.sentry:sentry-android:7.18.0` (in `app/build.gradle.kts`)
- **Wrapper**: `android/app/src/main/java/com/myhealth/app/crash/CrashReportingService.kt`
- **Boot**: `MyHealthApp.onCreate` calls `CrashReportingService.bootstrapIfEnabled( … )`
- **DSN source** (in priority order):
  1. Env var `SENTRY_DSN` (dev shell)
  2. `BuildConfig.SENTRY_DSN` (injected at build time from env var or `gradle.properties`)
- **User toggle**: Settings → Privacy → "Send crash reports (Sentry)"

Wear OS

- Same Gradle dep declared in `wear/build.gradle.kts`
- The wrapper lives in the phone module and Wear can call it via the shared `core` library if needed
- Currently no boot init in `wear/MainActivity.kt` — most crashes will surface in the phone app anyway since the watch shares much of the data layer

Backend (Express)

- **Dependency**: `@sentry/node` (in `server/package.json`)
- **Middleware**: `server/middleware/sentry.js`
- **Boot**: `server.js` calls `Sentry.init` + `Sentry.requestHandler` first, then `Sentry.errorHandler` after all routes
- **DSN source**: `SENTRY_DSN` env var only
- **No-op safe**: middleware is a graceful shim — local dev / CI smoke runs work without `@sentry/node` installed

Expo (React Native)

- **Dependency**: `sentry-expo` (in `mobile/package.json`)
- **Wrapper**: `mobile/src/crash.js`
- **Boot**: Call `bootstrapIfEnabled({ release })` from `App.js`
- **DSN source**: `EXPO_PUBLIC_SENTRY_DSN` env var (set in `eas.json` or `.env`)
- **User toggle**: AsyncStorage flag `crashReportsEnabled` (wire a toggle in Settings screen)

---

4. Setup steps (one-time per project)

4.1 Create the Sentry project

1. Sign up at sentry.io/signup (free)
2. Create an org (e.g. `american-group-llc`)
3. Create **5 separate projects** (one per platform — keeps quota and stack-trace symbolication isolated):
   - `myhealth-ios` (platform: iOS)
   - `myhealth-android` (platform: Android)
   - `myhealth-wear` (platform: Android)
   - `myhealth-server` (platform: Node.js)
   - `myhealth-expo` (platform: React Native)
4. Each project gives you a **DSN** that looks like: `https://abc123@o12345.ingest.sentry.io/67890`

4.2 Add the DSN as a GitHub Secret

In Settings → Secrets and variables → Actions → New repository secret:

Secret	Value
`SENTRY_DSN_IOS`	iOS project DSN
`SENTRY_DSN_ANDROID`	Android project DSN
`SENTRY_DSN_WEAR`	Wear project DSN (same as Android is fine to share quota)
`SENTRY_DSN_SERVER`	Server project DSN
`SENTRY_DSN_EXPO`	Expo project DSN

4.3 Wire the secrets into release builds

In `release.yml`'s `build-android` job:

```
- name: Build phone APK + AAB
  working-directory: android
  env:
    SENTRY_DSN: ${{ secrets.SENTRY_DSN_ANDROID }}
  run: ./gradlew :app:assembleRelease :app:bundleRelease
```

In `build-ios`:

```
- name: Build MyHealth.app
  working-directory: ios
  env:
    SENTRY_DSN: ${{ secrets.SENTRY_DSN_IOS }}
  run: xcodebuild ...
```

For the backend: set `SENTRY_DSN` as an env var on whatever PaaS you deploy to (Heroku, Render, Fly, etc.).

For Expo: add `EXPO_PUBLIC_SENTRY_DSN` to your EAS build profile.

---

5. User-visible toggle

Already wired:

Platform	Where the toggle lives
iOS	Settings → Privacy → “Send crash reports”
Android	Settings → Privacy → “Send crash reports (Sentry)”
Expo	TODO: add toggle to mobile Settings screen

The toggle takes effect: - **Immediately** on iOS (SDK closes when user opts out at runtime) - **Next launch** on Android (SDK can’t be re-initialized in the same process) - **Next launch** on Expo (same constraint)

---

6. Testing the wiring

```
## iOS — flip the toggle on, then trigger a crash:
SentrySDK.crash()

## Android — flip the toggle on, then:
throw RuntimeException("test crash")

## Backend — flip the env var on, restart, then hit a route that throws:
curl http://localhost:4000/api/test-error  # add this route locally to test
```

Within ~30 seconds the event should appear in your Sentry dashboard.

---

7. What’s NOT done

- [ ] Expo Settings screen lacks a UI toggle (data layer is wired, just needs the React component)
- [ ] Wear OS Application class doesn’t yet call `bootstrapIfEnabled` (most wear crashes will surface in the phone app anyway since the data layer is shared via `:core`)
- [ ] Source maps / dSYM upload to Sentry is not automated (manual upload from Xcode / `gradle uploadSentryNativeSymbols` task) — set up the `sentry-cli` GitHub Action when you need symbolicated crashes

SIGNING

SIGNING.md

Signing — FitFusion

This document lists every secret used by the release pipeline and the exact code-signing wiring for each platform. Workflows degrade gracefully when secrets are absent (Android → unsigned APK; iOS → unsigned Simulator `.app.zip`; Desktop → unsigned binaries).

See the canonical secret reference in the umbrella repo:
https://github.com/AmericanGroupLLC/AmericanGroupLLC/blob/main/SECRETS.md

---

Android

Secret	Required	Purpose
`ANDROID_KEYSTORE_BASE64`	optional	Base64-encoded `upload.jks`
`ANDROID_KEYSTORE_PASSWORD`	optional	Keystore password
`ANDROID_KEY_ALIAS`	optional	Key alias
`ANDROID_KEY_PASSWORD`	optional	Key password
`PLAY_STORE_SERVICE_ACCOUNT_JSON`	optional	Google Play upload
`PLAY_STORE_PACKAGE_NAME`	optional	Play Console package

A template is at `keystore.properties.example`.

---

iOS

The build-ios job takes one of two paths based on whether `APPLE_CERTIFICATE_BASE64` is set:

- **Set** → import .p12 to a temp keychain, install the provisioning profile, archive `-sdk iphoneos`, generate `exportOptions.plist` from `release.config.json.bundleId` (overridable via `APPLE_BUNDLE_ID`), `xcodebuild -exportArchive`, attach `<AppSlug>-iOS-<version>.ipa` to the GitHub Release.
- **Unset** → unsigned `-sdk iphonesimulator` build attached as `<AppSlug>-iOS-<version>-Simulator.app.zip` (Simulator-only, cannot run on real devices).

Secret	Required	Purpose
`APPLE_CERTIFICATE_BASE64`	optional	.p12 (Distribution)
`APPLE_CERTIFICATE_PASSWORD`	optional	.p12 password
`APPLE_KEYCHAIN_PASSWORD`	optional	Temp keychain pw
`APPLE_PROVISIONING_PROFILE_BASE64`	optional	Wildcard mobileprovision (recommended for MyHealthLiveActivity, MyHealthWidget, MyHealthMessages)
`APPLE_TEAM_ID`	optional	10-char team ID
`APPLE_BUNDLE_ID`	optional	Override the default bundle id (`com.fitfusion.ios`)
`APP_STORE_CONNECT_API_KEY_ID`	optional	TestFlight upload
`APP_STORE_CONNECT_API_ISSUER_ID`	optional	TestFlight upload
`APP_STORE_CONNECT_API_KEY_P8_BASE64`	optional	TestFlight upload

Provisioning profile shape

The default bundle ID is `com.fitfusion.ios`. Because this app ships with extensions (`MyHealthLiveActivity, MyHealthWidget, MyHealthMessages`), use a **wildcard** profile such as `com.fitfusion.ios.*` so the profile covers the host app and every extension target. App Store Connect requires explicit IDs for each target, but the *upload step* is gated by a separate secret and only runs after the .ipa is built.

Desktop

The desktop build job runs on Ubuntu and produces:

- Windows NSIS installer `*.exe` (signed only if you add Windows Authenticode in a follow-up — out of scope here).
- Linux AppImage (unsigned, portable, no install required).
- macOS .dmg (**unsigned**; Gatekeeper will block on first launch. See `STORE-PACKAGING.md` for the `xattr -cr` bypass and the notarization roadmap).

---

watchOS

The release job builds watchOS as a **standalone Simulator .app.zip** for QA. Submitting a watch app to the App Store requires the watch target to be embedded inside the iOS `.ipa` (watch2-app pattern), which kicks in automatically once `APPLE_CERTIFICATE_BASE64` is set and the Xcode project's iOS target depends on the watch target.

STORE-PACKAGING

STORE-PACKAGING.md

MyHealth — Store Packaging Reality Check

This is the **honest** doc covering how watch apps actually ship to each store and what the current MyHealth repo does vs. what each store requires.

TL;DR

Surface	What stores need	What this repo produces today	Gap
📱 iPhone	One `.ipa` containing the phone app	✅ `MyHealth-iOS-iPhone-vX-Simulator.app.zip` (sim) + `.ipa` via TestFlight job	none — once the ASC API key secret is set
⌚ Apple Watch	The watch app **embedded inside** the iPhone app's `.ipa`	❌ Currently builds a **standalone** `MyHealth-Apple-Watch-vX-Simulator.app.zip` from a separate `watch/HealthAppWatch.xcodeproj`	**Real gap** — see §1 below
🤖 Android phone	One `.aab` for Play Console	✅ `MyHealth-Android-Phone-vX.aab`	none
⌚ Wear OS	Either (a) standalone `.aab` with own Play listing, OR (b) embedded as wearable APK in phone AAB	✅ Standalone `MyHealth-Android-Watch-vX.apk` (route a)	optional — can also bundle into the phone AAB (§2)
🖥 Backend	Tarball / Docker image	✅ `MyHealth-Server-vX.tgz`	none
🌐 Web	Static zip / Pages deploy	✅ `MyHealth-Web-vX.zip` + Pages	none

---

1. Apple Watch — must be embedded in the iPhone `.ipa`

What Apple requires

> A watchOS app is shipped as a **single-target watchOS application embedded inside the iPhone app's bundle**. App Store Connect accepts ONE `.ipa` per app, and that `.ipa` contains the iPhone

> executable + all its embedded extensions including the watchOS app. Users install the iPhone app, and the Watch app appears automatically on the paired Apple Watch. [HIG · Distribute watchOS]

> **Note (watchOS 7+):** the legacy two-target structure (WatchKit App + WatchKit Extension) is deprecated. Modern watchOS apps are a **single SwiftUI target** with `WKApplication` declared in `Info.plist`. The `watchapp2` product type from earlier xcodegen docs is for the legacy structure and will produce build warnings on Xcode 15+.

What this repo does today

Two **separate** xcodegen projects:

```
ios/project.yml      → produces FitFusion.app          (iPhone app)
watch/project.yml    → produces HealthAppWatch.app     (watch app, NOT bundled)
```

Two separate `.app` files. **Apple App Store will reject this.**

The fix path (modern single-target watchOS app, Xcode 15+)

Migrate the watchOS target into `ios/project.yml` as an **embedded watchOS application** of the iPhone target. Concretely:

1. Move `watch/HealthAppWatch/Sources/**` → `ios/HealthAppWatch/`. Make sure `Info.plist` lives at `ios/HealthAppWatch/Info.plist` and contains:

   ```
   <key>WKApplication</key><true/>
   <key>WKWatchOnly</key><false/>
   <key>WKCompanionAppBundleIdentifier</key>
   <string>com.fitfusion.ios</string>
   ```

2. Edit `ios/project.yml` (xcodegen 2.42+ syntax — verified against Apple-sample-code template):

```yaml
targets:

  FitFusion:                          # existing iPhone app
    type: application
    platform: iOS
    deploymentTarget: "17.0"
    sources:
      - path: FitFusion
    dependencies:
      - target: HealthAppWatch        # ← NEW: embed the watch app
        embed: true
        codeSign: true
    settings:
      base:
        PRODUCT_BUNDLE_IDENTIFIER: com.fitfusion.ios

  HealthAppWatch:                     # NEW: single-target watchOS app
    type: application                 # (NOT application.watchapp2)
    platform: watchOS
    deploymentTarget: "10.0"
    sources:
      - path: HealthAppWatch
    info:
      path: HealthAppWatch/Info.plist
      properties:
        WKApplication: true
        WKWatchOnly: false
        WKCompanionAppBundleIdentifier: com.fitfusion.ios
    settings:
      base:
        # Bundle identifier MUST be the iPhone bundle id + ".watchkitapp"
        # for App Store Connect to recognise it as a paired watch app.
        PRODUCT_BUNDLE_IDENTIFIER: com.fitfusion.ios.watchkitapp
        TARGETED_DEVICE_FAMILY: "4"    # 4 = Apple Watch
        SUPPORTED_PLATFORMS: watchsimulator watchos
        SDKROOT: watchos               # forces the watchOS SDK
```

Key rules the original snippet got wrong:

- `type: application.watchapp2` is the legacy product type — use plain `application` with `platform: watchOS` instead.
- Do NOT set both `deploymentTarget: "10.0"` and `WATCHOS_DEPLOYMENT_TARGET` — they collide. Use `deploymentTarget` alone; xcodegen translates it correctly per platform.
- Bundle id MUST match the pattern `<iPhone-bundle-id>.watchkitapp` or App Store Connect won't pair the watch app to the iPhone app.
- The iPhone target's `dependencies` entry needs `embed: true` AND `codeSign: true` — without them xcodebuild will produce an `.ipa` where the watch app isn't actually bundled.

- `TARGETED_DEVICE_FAMILY: "4"` is required (4 = Apple Watch). Without it Xcode silently treats the target as iOS.

3. Delete `watch/project.yml` (or keep it for QA-only standalone builds).
4. Run `xcodegen generate` inside `ios/`.
5. Verify the embed worked:

```
xcodebuild -project ios/FitFusion.xcodeproj -list
xcodebuild -project ios/FitFusion.xcodeproj \
  -scheme FitFusion \
  -destination 'generic/platform=iOS' \
  -archivePath build/MyHealth.xcarchive archive

# The watch app should appear inside the .xcarchive:
ls build/MyHealth.xcarchive/Products/Applications/FitFusion.app/Watch/
#  → HealthAppWatch.app
```

6. Export to `.ipa` with the standard `exportOptions.plist` (already in `release.yml`'s `publish-testflight` job). The resulting `.ipa` zipped contains the watch app bundled inside the iPhone app's `Watch/` folder.
7. Verify after export:

```
unzip -l ios/build/ipa/FitFusion.ipa | grep HealthAppWatch.app
# Should list HealthAppWatch.app inside Payload/FitFusion.app/Watch/
```

Effort

~30-45 min on a Mac with Xcode 15+. **Not done yet** — left as a follow-up because it needs hands-on Xcode validation that's hard to do from CI alone.

Until that's done

The current `MyHealth-Apple-Watch-vX-Simulator.app.zip` artefact is **only useful for**: - Manually running the watch app in Apple Watch Simulator for QA / screenshots - Code-review of the watch UI without firing up Xcode

It is **NOT** a binary you can submit to the App Store.

---

2. Wear OS — two valid distribution shapes (we use shape A)

Shape A — Standalone Wear OS app *(what we do today)*

- Wear app has its own package (`com.myhealth.wear`)
- Listed as a separate app in Play Console (form factor: Wear OS)
- Phone APK + Wear APK are completely independent — a user can install one without the other
- ✅ This is what the current repo produces (`MyHealth-Android-Watch-vX.apk`)

Shape B — Bundled wearable APK inside the phone AAB

- Phone APK contains the Wear APK as a wearable feature module
- Single Play Console listing, single AAB upload
- Wear Play Store auto-installs the wear feature when the phone app is installed on a paired watch
- Modern Play AAB feature delivery makes this nicer than the old Wear 2.x bundling

Which one to use?

- **Shape A** is best for **standalone Wear OS apps** that don't need a phone (the user can install MyHealth on their Pixel Watch without owning the phone app). MyHealth fits this — the watch tabs work even without the phone app installed.
- **Shape B** is required if your watch app **only makes sense paired with a specific phone app**. Not our case.

No code change needed. Both wear and phone apps already declare `com.google.android.wearable.standalone = true` in their manifests, which is the required marker for Shape A.

To list both in Play Console

1. In Play Console, create **two separate apps**:
    - `com.myhealth.app` (Phone)
    - `com.myhealth.wear` (Wear OS)
2. Mark the Wear OS app with the "Wear OS" form factor in Store presence
3. Upload both AABs / APKs through the existing release pipeline (the `r0adkll/upload-google-play` step takes whatever AAB it finds — extend the matrix to also upload the wear APK to its own listing)

---

3. Updated `release.yml` matrix — what we *should* aim for

Asset	Today	Target after watch-embedding fix
`MyHealth-iOS-iPhone-vX.ipa`	⚠️ unsigned `.app.zip` only	✅ signed `.ipa` containing iPhone + watch (uploaded to TestFlight by `publish-testflight`)
`MyHealth-Apple-Watch-vX-*.app.zip`	✅ standalone QA artefact	🟡 keep for QA but it's no longer the store-shippable binary
`MyHealth-Android-Phone-vX.aab`	✅ ready for Play	✅ same
`MyHealth-Android-Watch-vX.apk`	✅ ready for Play (separate listing)	✅ same
`MyHealth-Server-vX.tgz`	✅	✅
`MyHealth-Web-vX.zip`	✅	✅

---

4. Action items before App Store submission

- [ ] Migrate `watch/` into `ios/project.yml` as a `watch2-app` target (§1)
- [ ] Verify the resulting `.ipa` contains both `FitFusion.app` and `HealthAppWatch.app` via `unzip -l MyHealth-iOS-iPhone-vX.ipa`
- [ ] Set `APP_STORE_CONNECT_API_KEY_*` secrets so `publish-testflight` uploads the combined `.ipa` automatically

5. Action items before Play Store submission

- [ ] Pay $25 + create two Play Console apps (`com.myhealth.app`, `com.myhealth.wear`) — only the **first** AAB upload needs to be manual; everything after that flows through CI
- [ ] Set `PLAY_STORE_SERVICE_ACCOUNT_JSON` + `PLAY_STORE_PACKAGE_NAME` secrets — the existing `publish-play-store` job auto-uploads the phone AAB
- [ ] Optionally add a second `publish-play-store` job step for the wear APK using a second package name secret

---

6. Honest summary

Question	Answer
Can I download today's `v1.0.1` binaries and use them?	✅ Yes — Android phone APK / AAB + Wear APK are real installable binaries
Can I submit `v1.0.1` to App Store today?	❌ No — the Apple Watch app is a separate `.app`, not embedded
Can I submit `v1.0.1` to Play Store today?	✅ Yes for the phone AAB; ✅ Yes for Wear APK as a separate listing
Will future tags ship a complete iOS `.ipa` with the watch inside?	⏳ Only after the §1 migration + `APP_STORE_CONNECT_API_KEY_*` secrets

The repo is **fully production-ready for Google Play** today (modulo the $25 fee + the one-time manual first upload). It's **80% production-ready for Apple App Store** — only the watch-embedding restructure is missing.

Desktop binaries (Electron)

The `build-desktop` job in `.github/workflows/release.yml` uses `electron-builder` on an Ubuntu runner to produce three artifacts attached to every GitHub Release:

Artifact	Signed?	Notes
`FitFusion-Setup-X.Y.Z.exe` (NSIS)	no	Windows SmartScreen will prompt on first run; click *More info → Run anyway*.
`FitFusion-X.Y.Z.AppImage`	no	`chmod +x` then double-click on Linux.
`FitFusion-X.Y.Z.dmg`	**no**	macOS Gatekeeper will refuse. To install:

```
xattr -cr "/Volumes/FitFusion/FitFusion.app"
cp -R "/Volumes/FitFusion/FitFusion.app" /Applications/
```

(or *System Settings → Privacy & Security → Open Anyway*.)

A future change can add a separate `macos-latest` job + Apple Developer cert to produce a properly notarised .dmg without restructuring this workflow.

The Electron shell loads the existing root `index.html` in a `BrowserWindow` with `contextIsolation` enabled. See `desktop/main.js`.

TESTING

TESTING.md

MyHealth — Sanity Testing Guide

This is the end-to-end checklist a tester walks through to verify each feature is working. Now covers **5 platforms** (iOS · watchOS · Android phone · Wear OS · Web/Expo) and **Guest Mode**.

---

0. Prerequisites

Surface	Needs
Marketing site	A web browser
Backend	Node.js ≥ 18
Expo (mobile)	Node + the Expo Go app on your phone (iOS or Android)
iOS / watchOS native	macOS · Xcode 15+ · `xcodegen` (`brew install xcodegen`) · iCloud account signed in on the simulator/device · Apple Developer team ID (free or paid)
Real-device-only features	A real iPhone (iOS 17+) and ideally a paired Apple Watch (watchOS 10+)

---

1. Backend (any OS)

```
cd server
cp .env.example .env
## edit .env to set JWT_SECRET to a random 32-char string
npm install
npm run dev          # → ✅ MyHealth API listening on http://localhost:4000
```

In a second terminal:

```
cd server
npm run smoke        # auto-registers a test user, exercises every route
```

Expected output: ~10 ✅ lines and `10 / 10 passed.`

Or run the Jest suite:

```
npm test
```

Manual curl spot-checks

```
curl http://localhost:4000/api/health-check
## {"ok":true,"service":"MyHealth API","time":"…"}

curl -X POST http://localhost:4000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Test","email":"t@t.com","password":"password123"}'
## {"user":{…},"token":"…"}

## (use returned token below)
TOKEN=…
curl -H "Authorization: Bearer $TOKEN" http://localhost:4000/api/profile
```

---

2. Marketing site (any OS, no install)

Just open `index.html` in any browser.

Section	Verify
Hero	Title bar reads “MyHealth · Your Personal Fitness OS”
Navbar	Logo says “MyHealth”; gradient buttons render
Unified Vision	“One MyHealth, Two Devices” heading
Pillars	3 cards (Physical / Mental / Emotional)
Top Apps	Filter tabs (All / Fitness / Running / Diet / Mental / Tracking) toggle the grid
iPhone vs Watch	Comparison table renders
Footer	Links to WHO + CDC open in new tab; copyright reads MyHealth

---

3. Expo cross-platform app (mobile or web)

```
cd mobile
npm install
npm start
```

- Press `w` for web, `i` for iOS sim, `a` for Android emulator, or scan the QR with **Expo Go** on a physical phone.
- Should land on the Login screen.
- Register → Profile → Log a metric → see it in History.

---

4. iOS native app — simulator (Mac required)

```
cd ios
xcodegen generate
open FitFusion.xcodeproj
```

In Xcode: 1. Select scheme **FitFusion** + an iPhone simulator (e.g. iPhone 15 Pro). 2. ▶️ Run.

Sanity matrix

Tab / surface	Steps	Expected
Login	Register a new account	Lands on Home tab
Home	Tap **How are you?** card	2-axis State of Mind picker opens
Home	Tap **Today's Suggested Workout**	Routes to Workout Detail with confidence + rationale
Train hub	6 tiles render	Anatomy / Library / Programs / Stretching / Custom / Send to Watch
Train → **Anatomy**	Tap a chest region on the front silhouette	Filtered list shows Bench Press, Push-up, etc.
Anatomy	Toggle **Back** in segmented picker	Lats, traps, lower back regions render
Train → **Library**	Search "deadlift"	Deadlift row appears
Library	Filter chips	Equipment / Level / Stretches toggle filter results
Train → **Programs**	Open Push/Pull/Legs → Push A	Shows 6 exercises linked to detail pages
Train → **Stretching**	List	10 stretches render
Train → **Custom**	Add 3 exercises → Save	Saved entry appears below
Exercise detail	Tap **Log a Set**	Logger opens; defaults to last session shape if any
Logger	Add 3 sets → Save	Returns to detail; PR badge updates if a new PR
Logger	Tap **Start rest timer**	Circular countdown sheet animates
Run	Tap a past run	RunDetailView; map renders if route exists
Eat	Snap Meal	Picker opens; classifier returns empty list (no `.mlmodel`) — fallback works
Eat	Read Label	OCR opens; pick photo of nutrition label → form fills with kcal/protein/etc.

Tab / surface	Steps	Expected
Eat	Search	Open Food Facts text search returns hits
Sleep	Open Wind Down	Breath circle animates; “Saved a 5-minute mindful session” after 5 min
Sleep	Tap **Log State of Mind** chip in Wind Down	StateOfMindLogger opens
Social	Add Friend	Friend appears in list
Social	Create Challenge	Active Challenges section renders new entry
Logout	Tap top-left logout icon	Returns to Login

Known simulator limitations

- **HealthKit reads** return zeros unless you seed the simulator’s Health app via macOS.
- **Cameras** unavailable → Snap Meal / Read Label / Barcode are no-ops; use the PhotosPicker library path with a saved nutrition-label image to test OCR.
- **Workout Mirroring** + **Live Activity** + **SharePlay** require real devices.

---

5. watchOS app — simulator (Mac required)

```
cd watch
xcodegen generate
open HealthAppWatch.xcodeproj
```

Pick **HealthAppWatch** + an Apple Watch simulator (Series 10, watchOS 11), ▶️.

Sanity matrix (vertical-paged tabs)

Pane	Steps	Expected
Login	Sign in with the same account	Lands on Quick Log
Quick Log	Tap +250 ml	Haptic, "✓ Logged 250 ml" toast
Live Workout	Start	Timer starts; HR shows 0 in sim (real device only)
Run	Start	Distance starts incrementing (sim won't move; pedometer fallback used)
Anatomy ← new	Tap **Upper Body**	List of upper-body muscles
Anatomy	Tap **Chest**	Filtered exercise list (Bench, Push-up, etc.)
Anatomy	Tap **Bench Press**	Detail with steps + form tips + Quick Log button
Anatomy	Tap **Quick Log** → set reps/weight via Digital Crown → Save	Haptic, sheet auto-dismisses, log persisted to CloudKit
Water / Weight	Crown to value → Save	Haptic, posted to backend + HealthKit
Mood	Tap an emoji	Haptic, posted as both `mood` and `state_of_mind_valence` metrics
History	Pull list	Recent metrics with icons + timestamps
Settings	Edit API URL	Saves to UserDefaults; restart picks up the new base URL

---

6. Real-device-only features

You need **at least one paired iPhone + Apple Watch** running iOS 17+ / watchOS 10+ and signed-in iCloud account.

Workout Mirroring (the moat)

1. Start a workout on the watch (Live Workout pane).
2. Within ~2 seconds the iPhone should auto-present `MirroredWorkoutView` (full-screen orange/pink/purple gradient with HR + calories + elapsed).

3. End the workout on the watch → iPhone sheet dismisses.

Live Activity

1. Same scenario as above.
2. Lock the iPhone → Lock Screen shows the workout banner with HR + elapsed.
3. On a Dynamic Island device → press-and-hold for the expanded variant.

Widgets

1. Long-press an empty Lock Screen / Home Screen spot → Add Widget.
2. Pick MyHealth → choose Readiness, Today's Plan, or Macro Rings.
3. Widget should populate within one refresh cycle (~1 minute) with values from the App Group (`group.com.fitfusion`).

GPS run

1. Watch → Run → Start.
2. Walk outside ≥1 km.
3. End run → on iPhone, RunDetailView shows polyline + 1+ split.

State of Mind ↔ Apple Health

1. iPhone → Home → How are you? → drag dot → Save.
2. Open the system Health app → Browse → State of Mind → entry appears.

CloudKit sync

1. Log a workout set on iPhone.
2. Within ~30s, the same exercise on the watch shows the new set in "Last session".

SharePlay shared workout

1. FaceTime a friend running MyHealth.
2. Open Train → Send to Watch → tap a workout.
3. The shared SharePlay banner appears in FaceTime; both devices schedule the same `WorkoutTemplate`.

iMessage Activity Card

1. After a workout ends, iPhone → Messages → MyHealth iMessage app → tap card.
2. Card inserts into compose field.

---

7. Automated tests

Suite	How to run	What it covers
Backend (Jest)	`cd server && npm test`	Auth, profile, metrics, social CRUD
FitFusionCore (XCTest)	`cd shared/FitFusionCore && swift test` *(macOS)*	ExerciseLibrary, Programs, NutritionLabelOCR, AdaptivePlanner heuristic, Codable round-trips

---

8. Known caveats — read before reporting bugs

- Bundle IDs are intentionally `com.fitfusion.*` even though the display name is MyHealth (preserves CloudKit data; see root README).
- `AdaptivePlanner.mlmodel` and `FoodClassifier.mlmodel` are **not** bundled; the heuristic / no-op fallbacks are intentional. See `ios/FitFusion/Models/README.md`.
- App icons fall back to Springboard placeholders until you drop a 1024×1024 PNG into `Assets.xcassets/AppIcon.appiconset/`. See the README in that folder.
- `DEVELOPMENT_TEAM` is empty in `project.yml`s — see `ios/SIGNING.md`.

---

9. Issue template

When something fails, please report:

```
Surface: iOS / Watch / Backend / Marketing / Expo
Device: iPhone 15 Pro Sim, iOS 17.5
Steps:
  1. Open Train tab
  2. Tap Anatomy
  3. ...
Expected: chest exercises listed
Actual: blank screen
Console:
  <copy-paste any Xcode console errors>
```

---

10. Guest Mode (no-account / local-only)

iOS

1. Reset the simulator (Device -> Erase All Content and Settings).
2. Build & run -> Login screen shows three actions: Sign In / Create an account / **Continue as Guest**.
3. Tap **Continue as Guest** -> 4-page Onboarding (Welcome -> Profile -> Goal -> Done).
4. Land on Home with no auth prompts. Open More -> Settings -> see Guest mode label.
5. Add a meal in Diary -> quit + relaunch -> meal still there (CoreData persisted).
6. From Settings tap **Sign in for cloud sync** -> Login sheet -> register -> app keeps the local data.

Android

1. Wipe the emulator -> fresh install.
2. Launch -> 4-page Onboarding (no login screen).
3. Bottom-nav: Home / Train / Diary / Sleep / More.
4. Add a medicine via FAB -> reminder fires at the chosen time.
5. Open More -> Settings -> Guest mode label.

Web (Expo)

1. cd mobile && npm start.
2. Login screen -> Continue as Guest -> lands on the app's home tab.

---

11. Android phone app

cd android ./gradlew :app:assembleDebug ./gradlew :app:installDebug # on a connected emulator/device

Surface	Steps	Expected
Onboarding	First launch	4-page flow -> Home
Home	Tap Vitals tile	Vitals screen renders
Vitals	Tap Biological Age card	Sliders update bio age in real time
Anatomy	More -> Anatomy -> tap a region tab	Muscle list scrolls
Medicine	More -> Medicines -> FAB -> fill name + dosage + 09:00 -> Save	Notification fires at 09:00 with Take / Snooze
Articles	More -> Health articles -> tap article	Detail body renders
Diary	More navigation -> Diary tab	Empty list (or recent meals if seeded)
Settings	More -> Settings	Imperial toggle persists across relaunches

Tests

./gradlew :core:testDebugUnitTest ./gradlew :app:testDebugUnitTest ./gradlew :app:connectedDebugAndroidTest

---

12. Wear OS app

cd android ./gradlew :wear:installDebug # on a Wear OS emulator (Wear 4 / API 33+)

Pane (swipe up to advance)	Verify
Quick Log	Renders header + subtitle
Live Workout	Renders, ready to wire HealthServicesGateway
Run	Same
Anatomy	ScalingLazyColumn lists every MuscleGroup with exercise count
Water / Weight / Mood / History / Settings	Render placeholder text

Watch face -> long-press -> add MyHealth Readiness complication. A ReadinessTileService is also registered (visible in the Tiles carousel).

---

13. Automated CI

Every push triggers GitHub Actions defined under .github/workflows/.

Workflow	Trigger	What it runs
ci.yml	push + PR	Backend Jest+smoke / Android core+app unit tests / iOS Swift Package tests + sim build / Watch sim build / marketing lint
android.yml	push to main + workflow_dispatch	Android instrumented tests on a real emulator (KVM-backed)
ios.yml	push touching ios/ watch/ shared/	iOS + watchOS sim builds (no signing)
backend.yml	push touching server/	Jest + 13/13 smoke run
marketing.yml	push touching index.html/styles.css/script.js	Deploys static site to GitHub Pages
release.yml	push of v* tag	Build everything, attach to GitHub Release, optional Play Store upload

14. Local one-shot test runner

./scripts/test-all.sh runs every suite available on the current OS:

- Backend : Jest + smoke
- Android : core + app unit tests (skipped if Gradle not installed)
- iOS+Watch: Swift Package tests + iOS sim build (skipped if not on macOS)
- Marketing: html-hint best-effort

15. Local simulator / emulator launchers

Script	Requires	What it does
./scripts/run-ios-sim.sh	macOS + Xcode + xcodegen	Boots iPhone 15 sim, builds, installs, launches MyHealth.app
./scripts/run-android-emulator.sh	ANDROID_HOME + a Pixel AVD	Boots phone emulator, installs, launches com.myhealth.app
./scripts/run-wear-emulator.sh	ANDROID_HOME + Wear AVD	Boots Wear emulator, installs, launches com.myhealth.wear

16. Release dry-run

./scripts/release-dry-run.sh v1.2.0 builds every artefact locally without pushing anything (mirrors what release.yml does on a tag push). Outputs land in distribution/staging-v1.2.0/.

17. Direct Google Play upload

Yes - the release.yml workflow uploads the AAB to Play Console automatically when secrets are set (PLAY_STORE_SERVICE_ACCOUNT_JSON + PLAY_STORE_PACKAGE_NAME). See RELEASING.md for the full setup.

---

18. CI test strategy for FitFusionCore (audit summary)

The shared Swift Package contains iOS-only / watchOS-only frameworks by design. Comprehensive audit of imports inside `shared/FitFusionCore/Sources/FitFusionCore/`:

File	Imports	macOS-host compatible?
AuthStore.swift / Models.swift / APIClient.swift	Foundation, SwiftUI	yes
HealthArticleSeed.swift	Foundation	yes
CloudStore.swift / FriendsStore.swift / ChallengesStore.swift	Foundation, CoreData, CloudKit, Combine	yes (macOS 13+)
BiologicalAgeEngine / BadgesEngine / StreaksEngine / Exercise / WorkoutProgram / LeaderboardClient / AdaptivePlanner	Foundation [+ CoreML]	yes
WatchConnectivity/Bridge.swift	Foundation, **WatchConnectivity**	NO (already gated with #if canImport)
PersonalFineTuner.swift	Foundation, CoreML, **BackgroundTasks**	partial (gated with #if os(iOS))
MealPhotoRecognizer.swift	Foundation, **UIKit**, Vision, CoreML	NO
NutritionLabelOCR.swift	Foundation, **UIKit**, Vision	NO

Because UIKit + WatchConnectivity cannot link on the macOS host, `swift test` on a macos-14 runner cannot test the package end-to-end. Two options exist:

A. Test on the iOS Simulator via xcodebuild. **(Chosen.)** B. Split FitFusionCore into a pure-Swift sub-module + an iOS-specific module.

The CI workflows now run:

`xcodebuild test -scheme FitFusionCore-Package -destination 'platform=iOS Simulator,name=iPhone 15,OS=latest'`

This natively builds the package against an iOS SDK (UIKit/WatchConnectivity both link) and runs the test target on the iPhone simulator. macOS host tests were dropped from Package.swift and the workflows.

19. Known CI caveats

- Codecov upload steps emit a warning when `CODECOV_TOKEN` is unset on a private repo — they are non-fatal (`fail_ci_if_error: false`).
- Marketing Pages deploy is gated on the configure-pages step succeeding; one-time enable at Settings -> Pages -> Source: GitHub Actions.
- release.yml Play Store upload is gated on `PLAY_STORE_SERVICE_ACCOUNT_JSON` secret presence; safe to commit before secrets exist.

WORKOUT-MEDIA

WORKOUT-MEDIA.md

MyHealth — Workout Media + Condition-Aware Suggestions

> **Doctor's advice always wins.** This document covers how MyHealth shows exercise GIFs, filters workouts by user-declared health conditions, and serves diet suggestions tuned to those conditions — without ever being a medical device.

---

1. Where the medical-decision line is drawn

Capability	What MyHealth does	Medical-device classification
Show a workout GIF	✅ generic instructional content	wellness app
Skip "back squat" when user declared low-back pain	✅ heuristic filter	wellness app
Show "DASH foods" when user declared hypertension	✅ public-health guidance	wellness app
Diagnose hypertension from BP readings	❌ never	would be SaMD
Recommend a specific medication or dosage	❌ never	strictly regulated
Detect arrhythmia from heart-rate data	❌ never	FDA-cleared territory

Per the FDA Software-as-a-Medical-Device guidance, MyHealth stays firmly in the **wellness** category by: - Never producing a diagnosis - Never recommending medication - Never claiming clinical accuracy - Showing a **doctor disclaimer** banner on every condition-driven screen

---

2. Health conditions the user can declare (opt-in)

Stored locally in UserDefaults / DataStore. Never sent to the server. Never logged via analytics. Always editable from **Settings** → **Health profile**.

Category	Conditions
Cardiovascular	High BP · Low BP · Heart condition
Metabolic	Type-1 diabetes · Type-2 diabetes · Obesity (BMI ≥ 30)
Respiratory	Asthma
Musculoskeletal	Knee · Ankle · Shoulder · Lower back · Osteoporosis
Other	Pregnancy · Kidney issue · Liver issue · Anemia
Default	None — full library available

---

3. Workout filtering rules

The shared `ExerciseMedia` map drives both platforms (iOS + Android use the same exercise IDs and same condition rules).

```
ExerciseLibrary.recommended(for: userConditions)
   ↓
   filter out anything in ExerciseMedia.cautions[exerciseId] ∩ userConditions
   ↓
   sort by overlap with ExerciseMedia.benefits[exerciseId] ∩ userConditions
   ↓
   return safe + tailored list
```

Examples

User condition	Hidden exercise	Why
Hypertension	Back Squat, Deadlift, OHP	Valsalva manoeuvre causes BP spike
Knee injury	Walking Lunge, Goblet Squat	Heavy knee flexion
Pregnancy	Crunches, Pullup, Heavy Squat, Downward Dog	Inversions, abdominal compression, fall risk
Osteoporosis	Heavy Squat, Deadlift	Spinal compression risk
Heart condition	Treadmill Run, Jump Rope	Until cleared by cardiologist

Beneficial exercises (boosted to top)

Condition	Exercises promoted
Lower back pain	Cat-Cow, Child's Pose, Hip Thrust, Hip Flexor Stretch
Hypertension	Rowing Machine (low-impact, controlled breathing)
Type-2 diabetes	Push-up, Dumbbell Press, Rowing Machine
Osteoporosis	Lateral Raise, Calf Raise (gentle weight-bearing)
Asthma	Neck Rolls, Doorway Pec Stretch (low-intensity)

---

4. Workout media (GIFs / videos)

Asset hosting

```
https://americangroupllc.github.io/HealthApp/assets/exercises/<exercise-id>.gif
https://americangroupllc.github.io/HealthApp/assets/exercises/<exercise-id>.jpg
```

Hosted on the existing **GitHub Pages** marketing site — zero extra infrastructure cost. The repo's `marketing.yml` workflow already deploys that site, so dropping new GIFs into `/assets/exercises/` ships them worldwide for free.

Why URLs instead of bundled assets

- **Lightweight install** — base APK / IPA stays under 25 MB
- **Updateable without app review** — fix a wrong GIF without shipping a new app version
- **Lazy-loaded** — only the exercises the user actually opens are downloaded
- **Graceful fallback** — if the GIF 404s, the SF Symbol / Material icon shows instead

Override via env var

Set `EXERCISE_MEDIA_BASE_URL` at build time (Xcode / Gradle) to point at a private CDN if you don't want to use GitHub Pages.

Where to source GIFs (free / CC-licensed)

Source	Licence	Quality
Wger.de exercise database	CC-BY-SA 4.0	Static images, decent coverage
Wikimedia Commons	Various OSS	Mixed
Pexels free workout videos	Pexels free licence	High quality, easy to convert to GIF
Record your own	–	Best fit for app brand

A small `scripts/fetch-exercise-gifs.sh` could pull from wger.de's API and drop the results into `assets/exercises/`. Not yet automated — manual upload works.

---

5. Diet suggestions

Pure on-device static map in `DietSuggestionsService`. For each condition, returns:

- **Pattern** — Mediterranean, DASH, low-GI, low-sodium, etc.
- **Foods to favor** — 6-10 specific items
- **Foods to limit** — 4-6 specific items
- **Daily targets** — sodium / fiber / protein / etc. caps
- **Notes** — 1-2 sentence rationale

Sources

The mappings are based on widely-published public-health dietary patterns: - DASH (NIH / NHLBI) for hypertension - Mediterranean (PREDIMED study) for heart + asthma - ADA standards for diabetes - KDOQI guidelines for kidney - ACOG guidelines for pregnancy - NIH ODS for osteoporosis

These are **public consensus guidance**, not personalized prescriptions. The disclaimer banner on every screen makes that explicit.

---

6. iOS implementation

File	Role
`shared/FitFusionCore/ … /Health/HealthConditions.swift`	enum + ObservableObject store
`shared/FitFusionCore/ … /Exercises/ExerciseMedia.swift`	URL catalogue + cautions/benefits maps
`shared/FitFusionCore/ … /Diet/DietSuggestionsService.swift`	static catalogue
`ios/FitFusion/Views/More/HealthProfileView.swift`	Settings → Health profile screen
`ios/FitFusion/Views/Diary/DietSuggestionsView.swift`	Diary tab → Diet suggestions screen

The existing exercise list views automatically benefit because `ExerciseLibrary.recommended(for: conditions)` filters at the data layer.

To embed a GIF in any exercise detail view:

```
AsyncImage(url: ExerciseMedia.gifURL(for: exercise.id)) { phase in
    switch phase {
    case .success(let image): image.resizable().aspectRatio(contentMode: .fit)
    default: Image(systemName: exercise.equipment.systemImage)
    }
}
.frame(maxHeight: 240)
```

---

7. Android implementation

File	Role
`android/core/ … /health/HealthConditions.kt`	enum + medical map (mirrors Swift)
(Settings screen toggle)	TODO — wire same way as crash-reports toggle
(Compose Coil GIF loader)	`AsyncImage(model = ExerciseMedia.gifUrl(id))`

To render a GIF in Compose:

```
val imageLoader = ImageLoader.Builder(LocalContext.current)
    .components { add(GifDecoder.Factory()) }
    .build()
AsyncImage(
    model = ExerciseMedia.gifUrl(exercise.id),
    contentDescription = null,
    imageLoader = imageLoader,
    placeholder = painterResource(R.drawable.ic_exercise_placeholder),
)
```

---

8. Doctor-disclaimer policy

Every screen that uses `HealthConditionsStore` data MUST display:

> ⚕ This is general guidance — not medical advice. Always check with your doctor before starting a new workout or diet plan.

iOS implementation: a 12-pt orange banner at the top of the screen (see `HealthProfileView` and `DietSuggestionsView`).

Android implementation: same banner using `MaterialTheme.colorScheme.tertiary`.

The store also tracks `lastDoctorReview` and prompts the user every 6 months to re-confirm their condition list.

---

9. What this is NOT

- ❌ Not an insulin-dosing calculator
- ❌ Not a heart-arrhythmia detector
- ❌ Not an ECG interpreter
- ❌ Not a meal-planner with calorie totals (we don't auto-build menus)
- ❌ Not a replacement for a physician, dietitian, or PT

Stay on the wellness side of the regulatory line, ship freely on App Store and Play Store as a fitness app, and clearly label every condition-driven suggestion with the doctor banner.